Deep Learning

Lecture on cnns
UG-3

Convolution

1D (continuous, discrete) :

$$f * g\ (x) = \int_{\alpha=-\infty}^{\infty} f(\alpha) g(x-\alpha) d\alpha$$

$$= \sum_{\alpha=0}^{N-1} f(\alpha) g(x-\alpha)$$

Input

Kernel

Output is sometimes called Feature map

2D (continuous, discrete) :

$$f * g\ (x,y) = \int_{\alpha=-\infty}^{\infty} \int_{\beta=-\infty}^{\infty} f(\alpha,\beta) g(x-\alpha, y-\beta) d\alpha d\beta$$

$$= \sum_{\alpha=0}^{N-1} \sum_{\beta=0}^{N-1} f(\alpha,\beta) g(x-\alpha, y-\beta)$$

Convolution Properties

- Commutative:

  $f*g = g*f$

- Associative:

  $(f*g)*h = f*(g*h)$

- Homogeneous:

  $f*(\lambda g)= \lambda\, f*g$

- Additive (Distributive):

  $f*(g+h)= f*g+f*h$

- Shift-Invariant

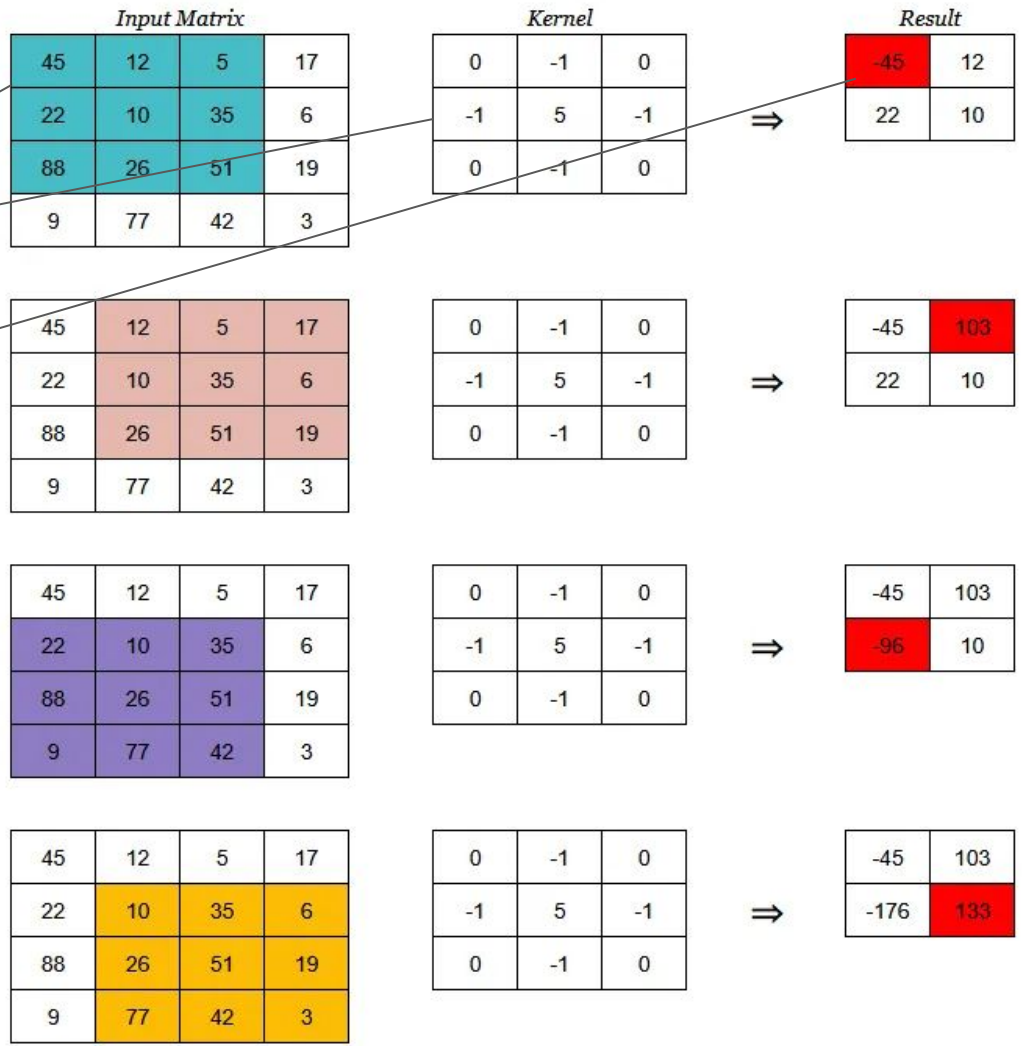
  $f*g(x-x0,y-yo)= (f*g)\,(x-x0,y-yo)$

The convolution operation

$45*0 + 12*(-1) + 5*0$

$+ 22*(-1) + 10*5 + 35*(-1)$ $= -45$

$+ 88*0 + 26*(-1) + 51*0$



Convolution in digital image processing

Original	Gaussian Blur	Sharpen	Edge Detection
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
			

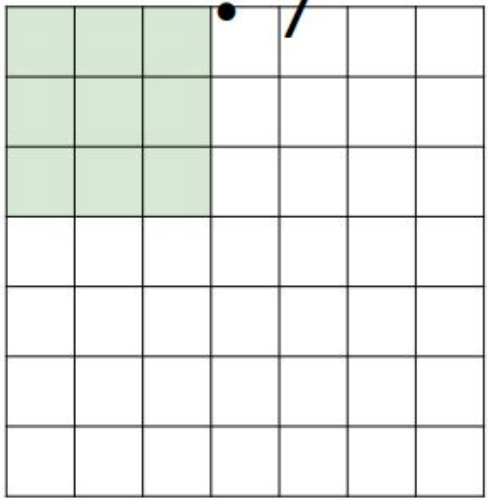
Traditional kernels are

- fixed,
- task-agnostic, and
- linear,

whereas CNN-learned kernels are data-driven, task-specific, adaptive, and composable across layers.

Convolutions: More detail

A closer look at spatial dimensions:



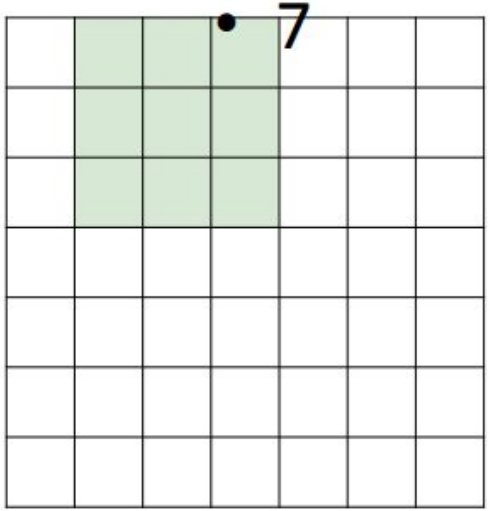
- 7

- 7x7 input (spatially) assume 3x3 filter

- 7

Convolutions: More detail

A closer look at spatial dimensions:



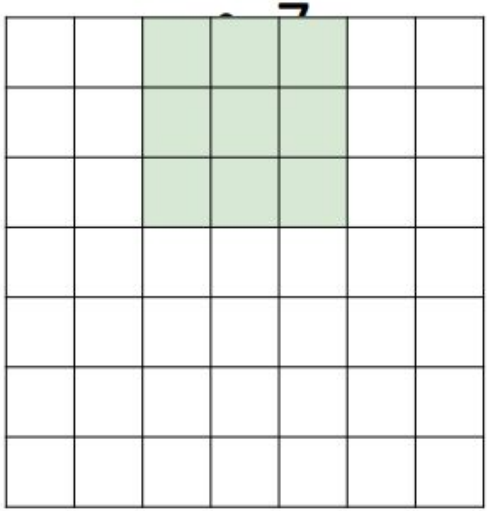
- 7

- 7x7 input (spatially) assume 3x3 filter

- 7

Convolutions: More detail

A closer look at spatial dimensions:

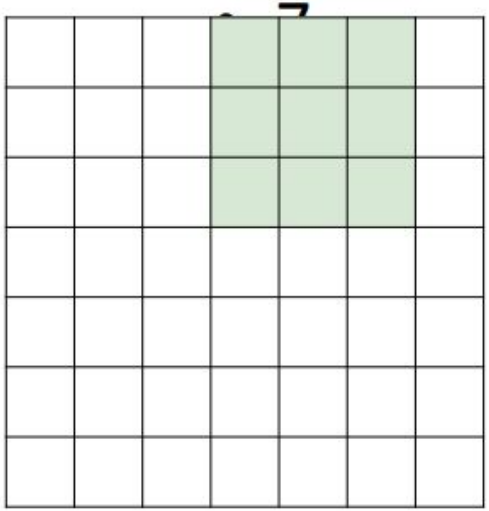


- 7x7 input (spatially) assume 3x3 filter

- 7

Convolutions: More detail

A closer look at spatial dimensions:

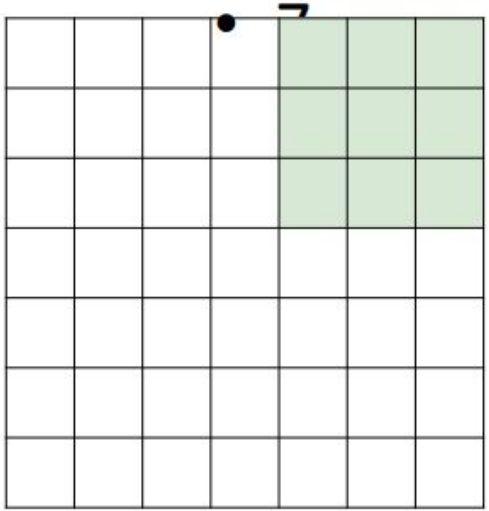


- 7x7 input (spatially) assume 3x3 filter

- 7

Convolutions: More detail

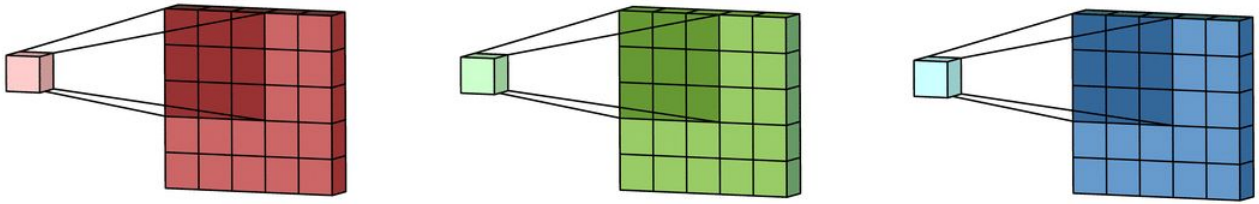
A closer look at spatial dimensions:

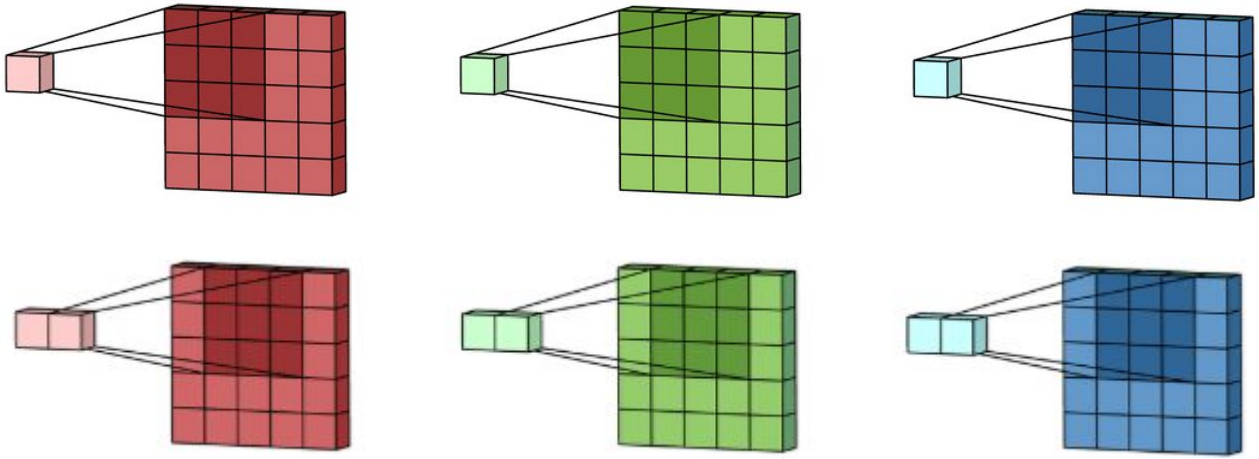


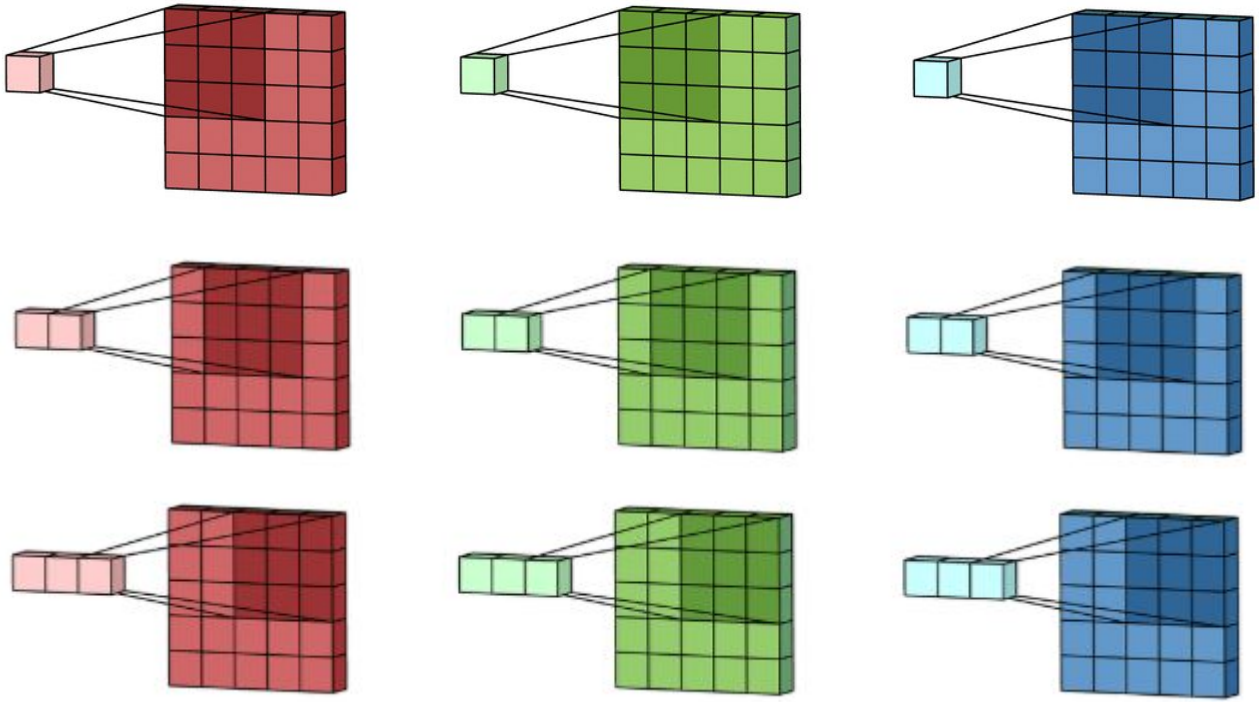
- 7x7 input (spatially)
  assume 3x3 filter

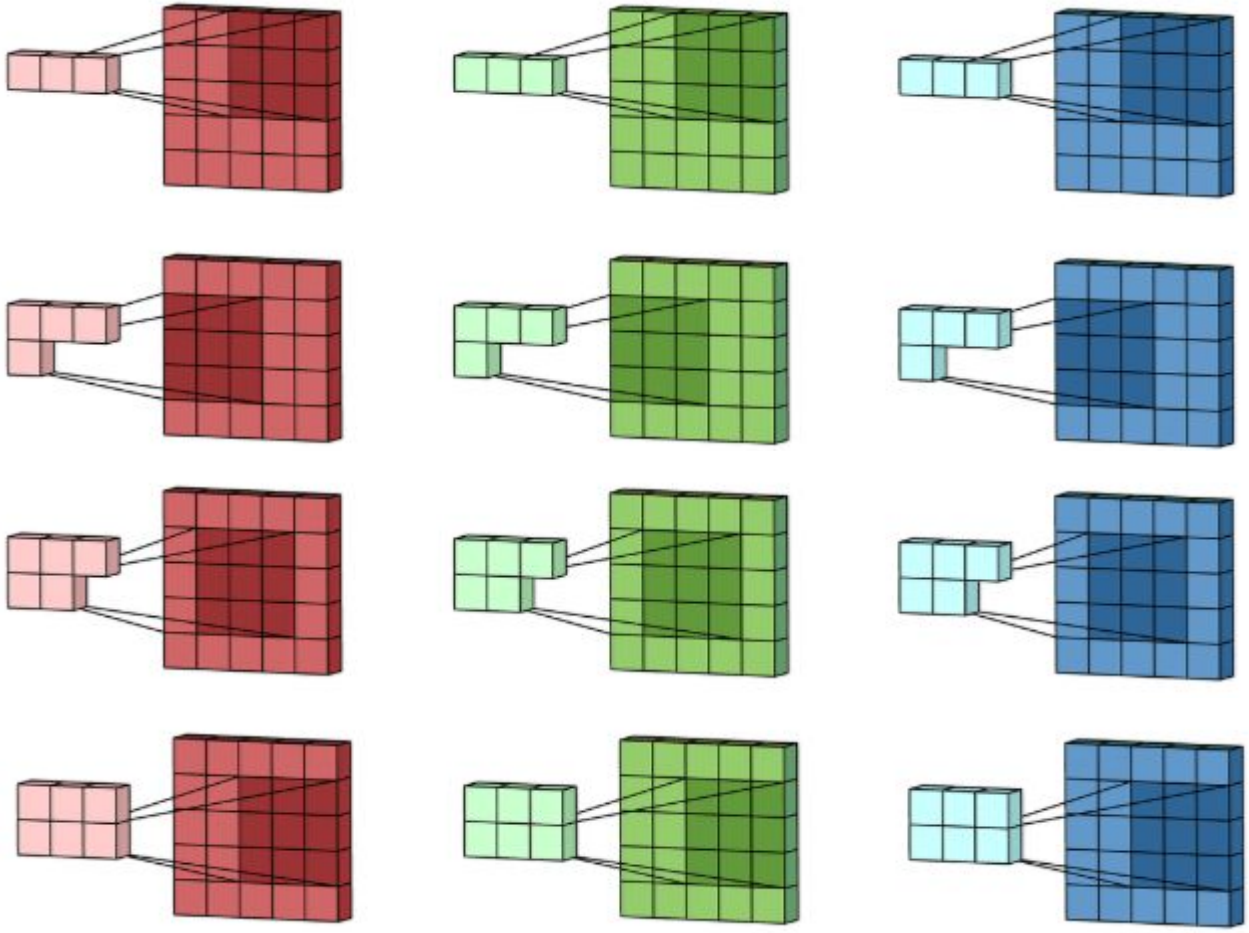
=> 5x5 output

Convolution in multi channel inputs

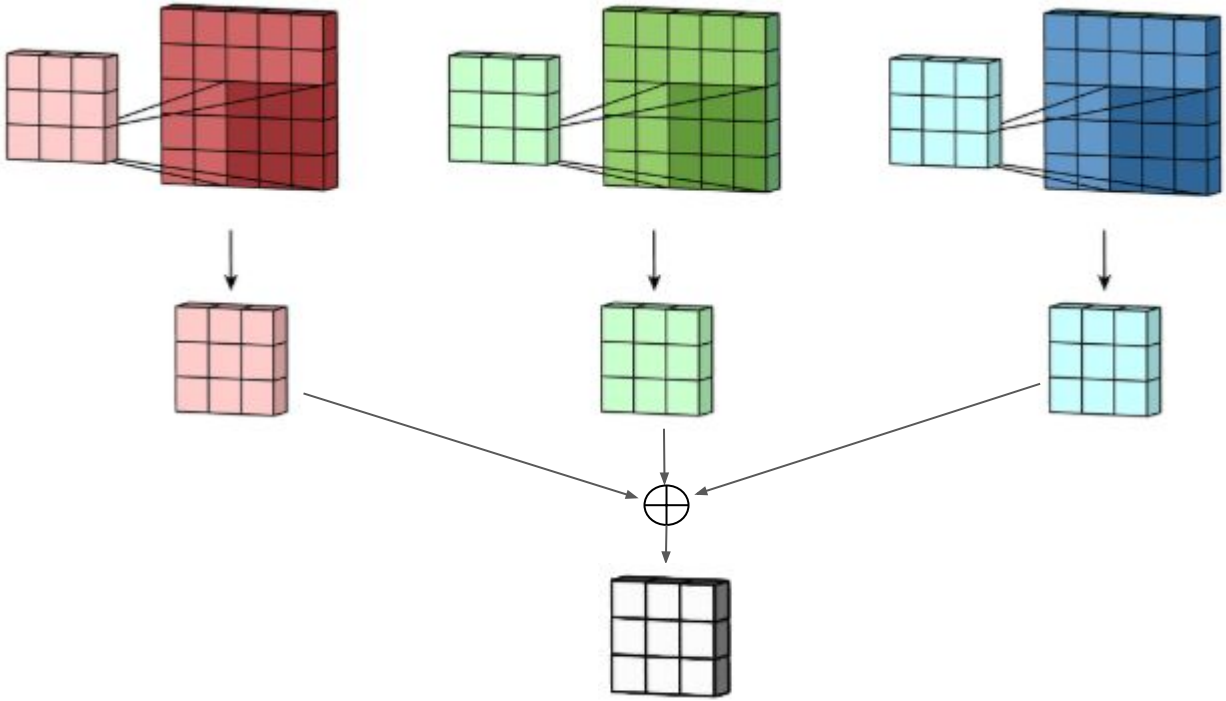






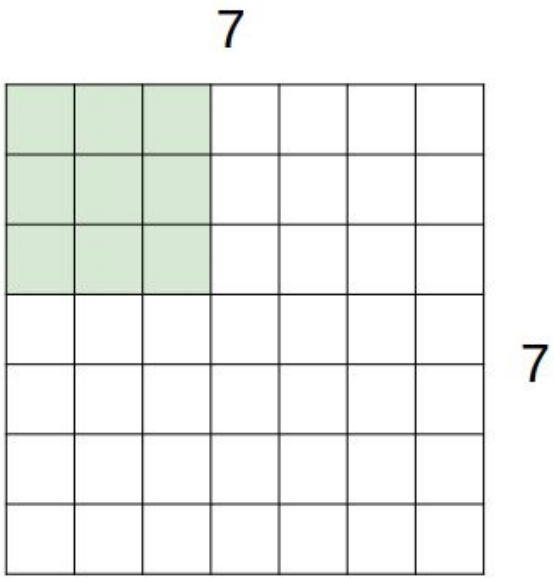


5x5x3 (RGB) image is convolved with 3x3x3 kernel to get a 3x3x1



Convolutions: More detail Stride

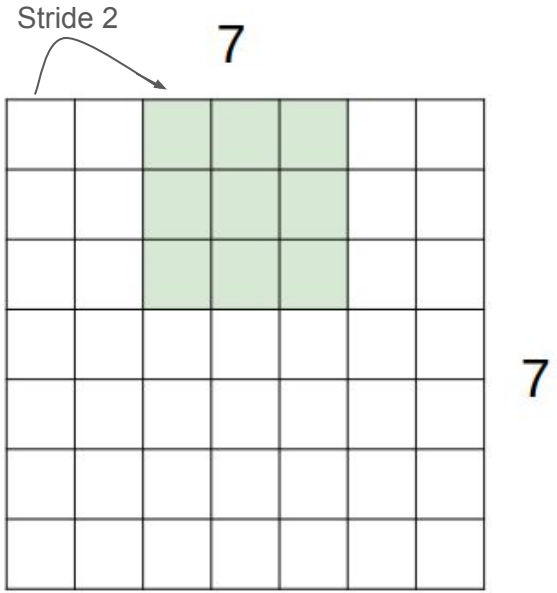
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Convolutions: More detail

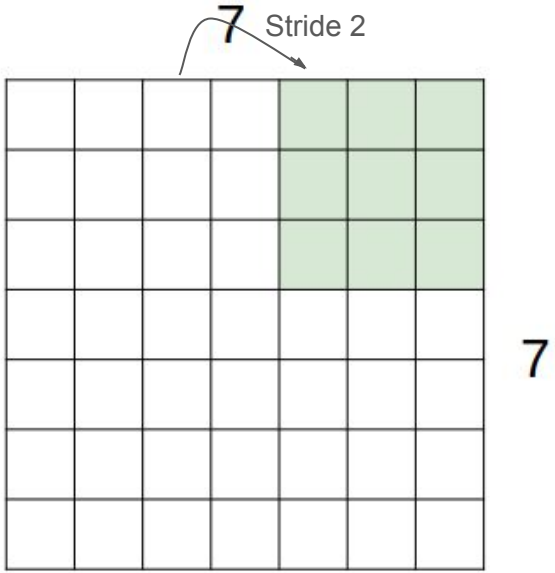
A closer look at spatial dimensions:



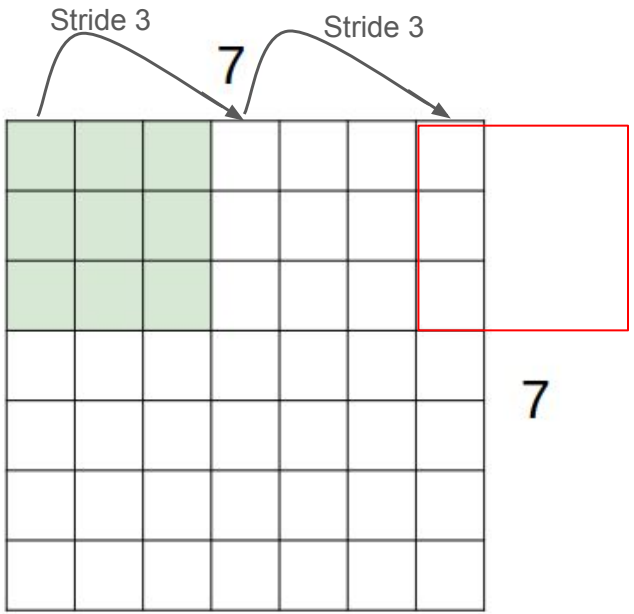
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Convolutions: More detail

A closer look at spatial dimensions:

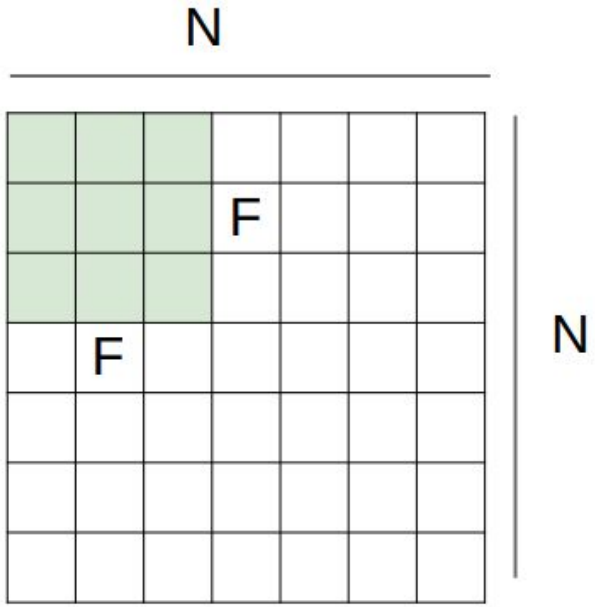


7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

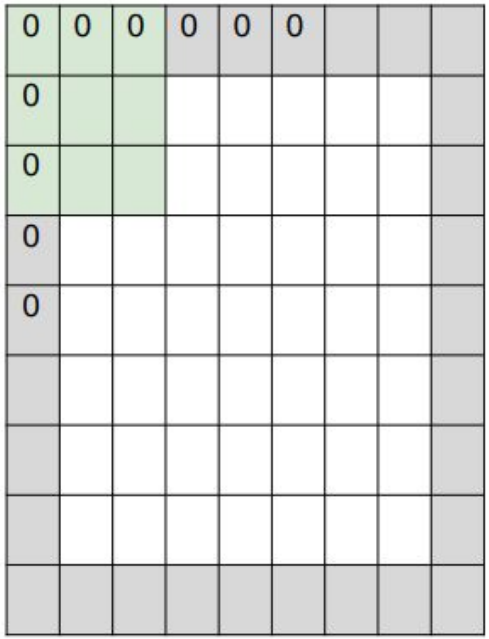
doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.



Output size:
(N - F) / stride + 1

e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33 :\

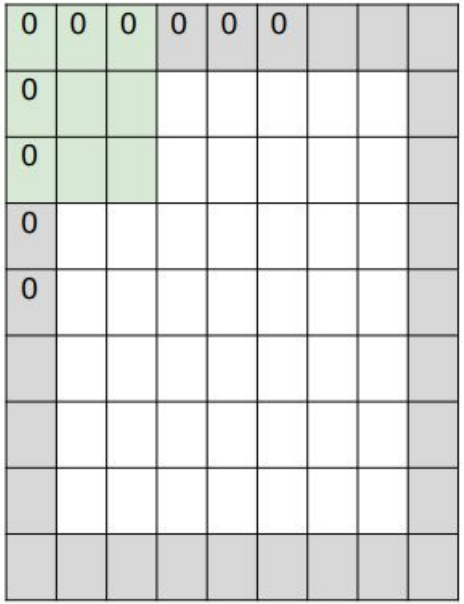
In practice: Common to zero pad the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

(recall:)
(N - F) / stride + 1

In practice: Common to zero pad the border

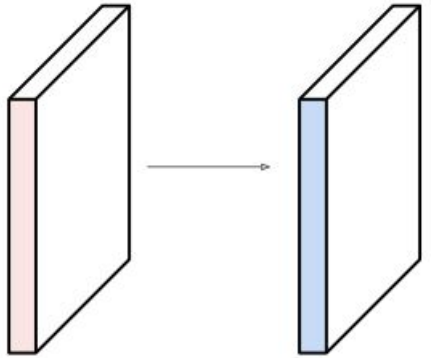


e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

(N + 2*padding - F) / stride + 1

Examples



Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

(Depth:3 is inherent => 5x5x3)

Output volume size:

(32+2*2-5)/1+1 = 32 spatially, so

32x32x10

Data flow in a Convolutional layer

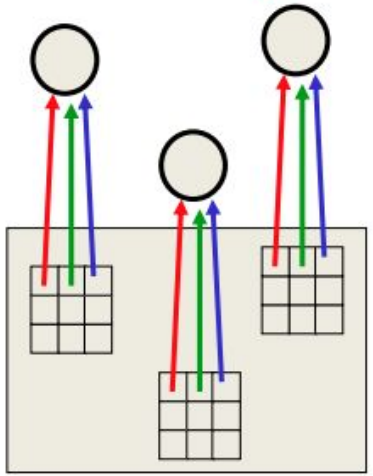
- Accepts a volume of size W1×H1×D1
- Requires four hyperparameters:
  - Number of filters K
  - their spatial extent F
  - the stride S
  - the amount of zero padding P
- Produces a volume of size W2×H2×D2
  - W2=(W1−F+2P)/S+1
  - H2=(H1−F+2P)/S+1
  - D2=K
- With parameter sharing, it introduces F·F·D1 weights per filter, for a total of (F·F·D1)·K weights and K biases.
- In the output volume, the d-th depth slice (of size W2×H2) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

The replicated feature approach

Why same kernel is used across all positions in an image/feature map?

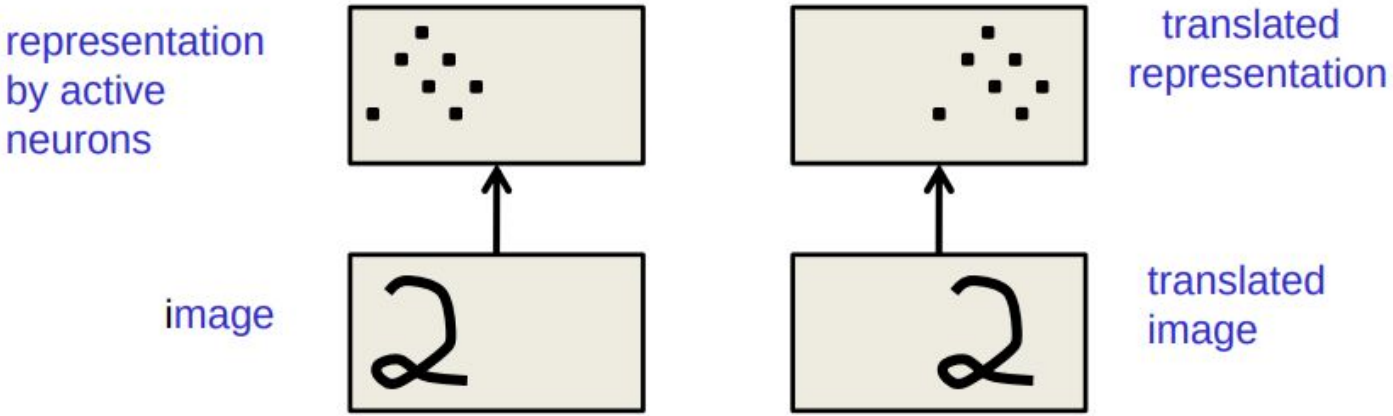
- Use many different copies of the same feature detector with different positions.
  - Could also replicate across scale and orientation (tricky and expensive)
  - Replication greatly reduces the number of free parameters to be learned.
- Use several different feature types, each with its own map of replicated detectors.
  - Allows each patch of image to be represented in several ways.

The red connections all have the same weight.



What does replicating the feature detectors achieve?

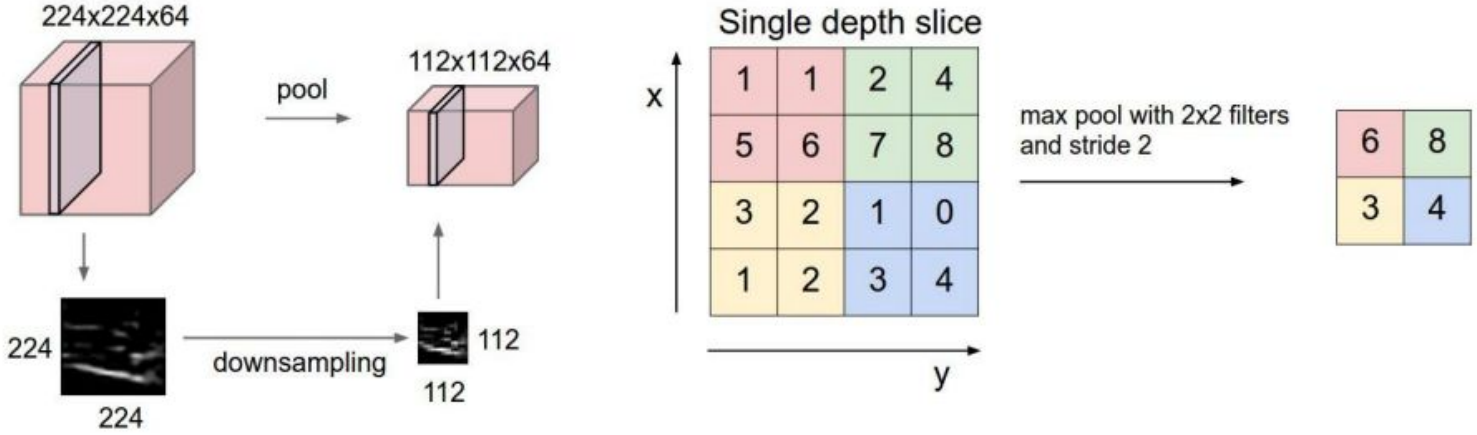
- Equivariant activities: Replicated features do not make the neural activities invariant to translation. The activities are equivariant.



- Invariant knowledge: If a feature is useful in some locations during training, detectors for that feature will be available in all locations during testing.

3. Spatial Pooling

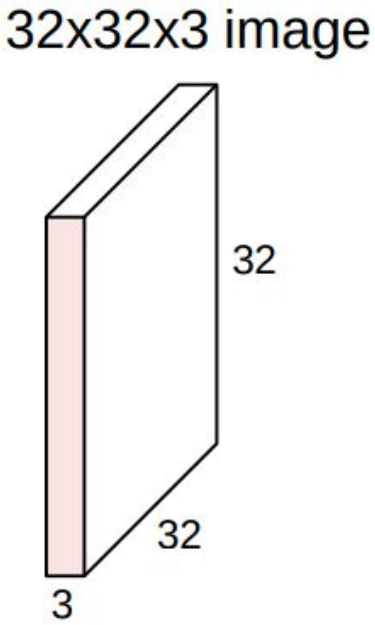
- Sum or max over non-overlapping / overlapping regions
- Role of pooling:
  - Invariance to small transformations
  - Larger receptive fields (neurons see more of input)



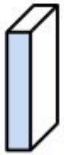
General pooling layer

- Accepts a volume of size W1×H1×D1
- Requires two hyperparameters:
  - their spatial extent F
  - the stride S
- Produces a volume of size W2×H2×D2 where:
  - W2=(W1−F)/S+1
  - H2=(H1−F)/S+1
  - D2=D1
- Introduces zero parameters
- Other pooling functions: Average pooling, L2-norm pooling

Preview of CNN

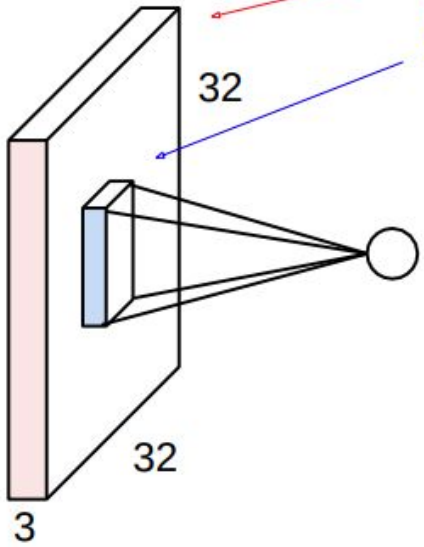


5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer



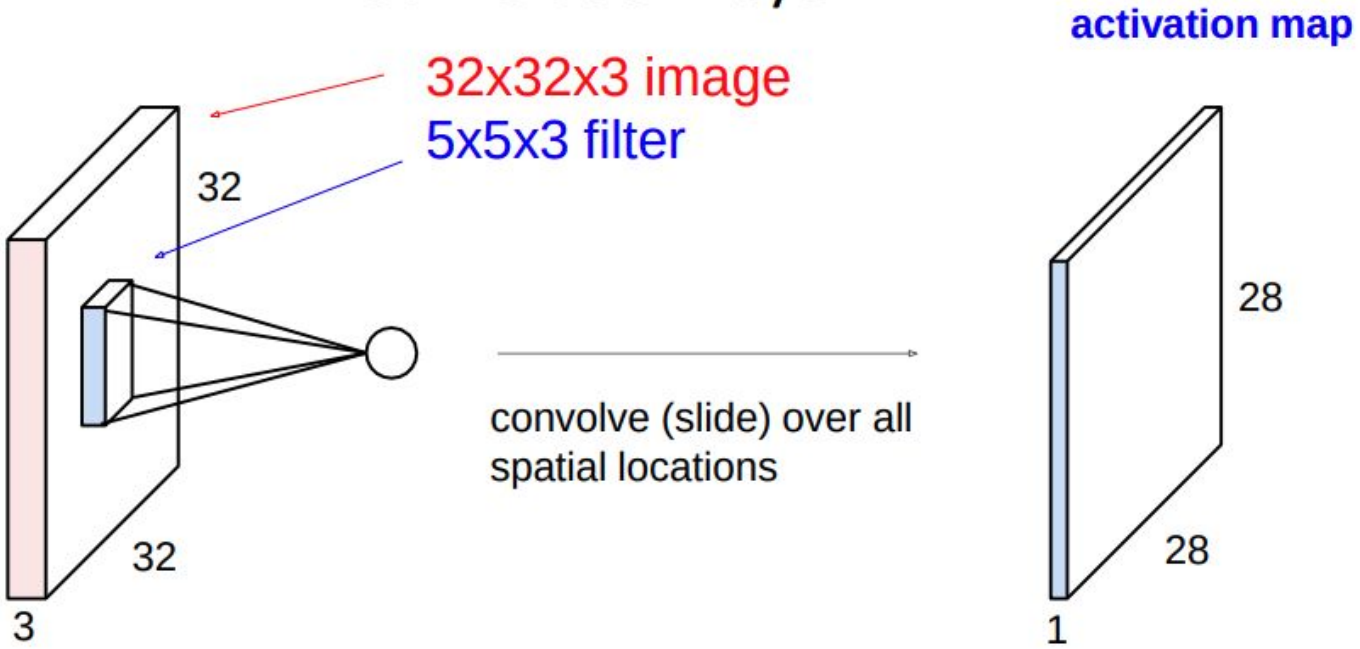
32x32x3 image

5x5x3 filter w

1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

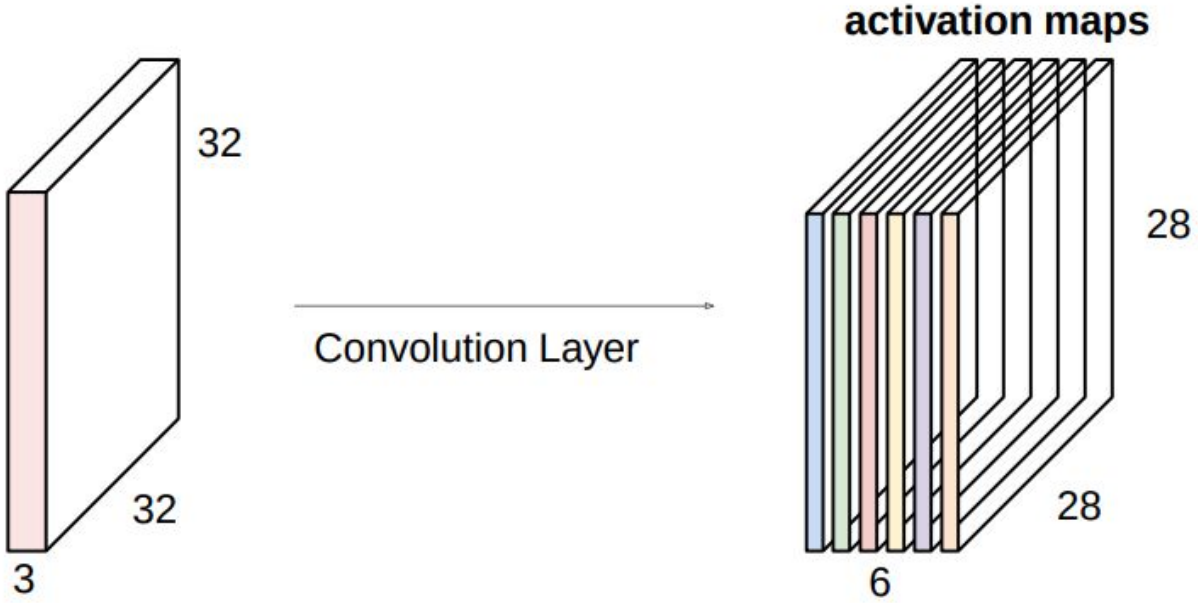
Convolution Layer



Convolution Layer

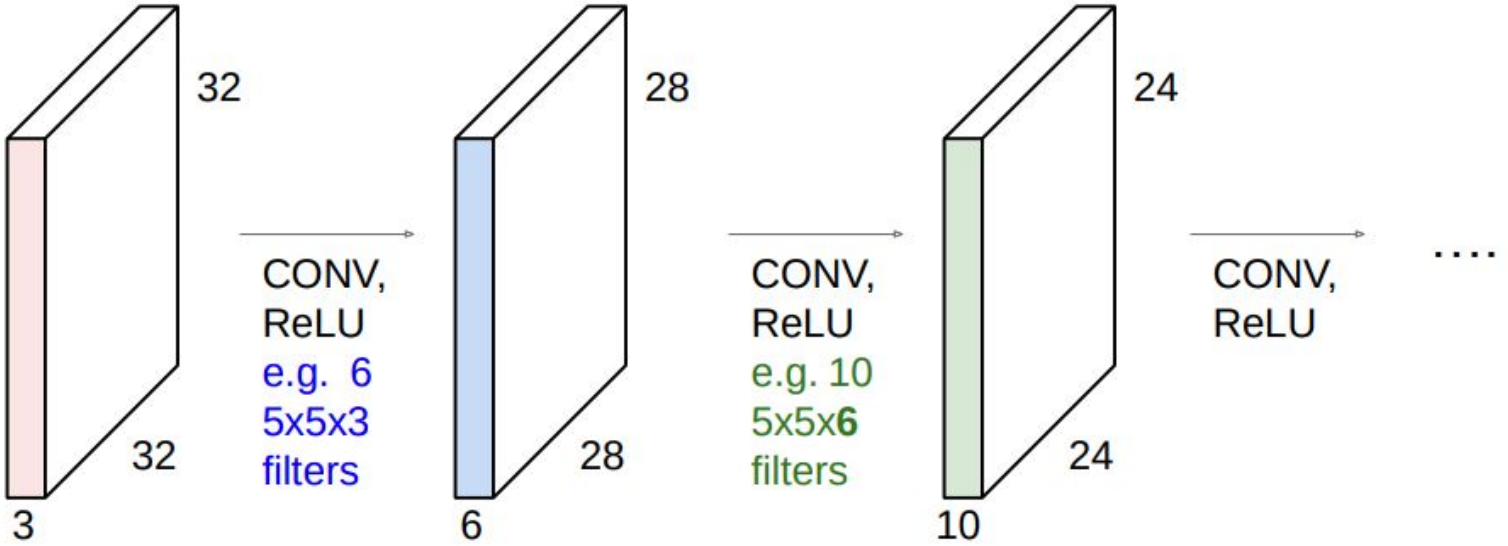
consider a second, green filter

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

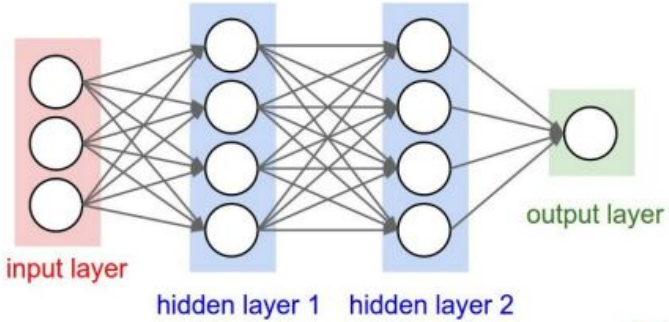


We stack these up to get a "new image" of size 28x28x6!

Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions

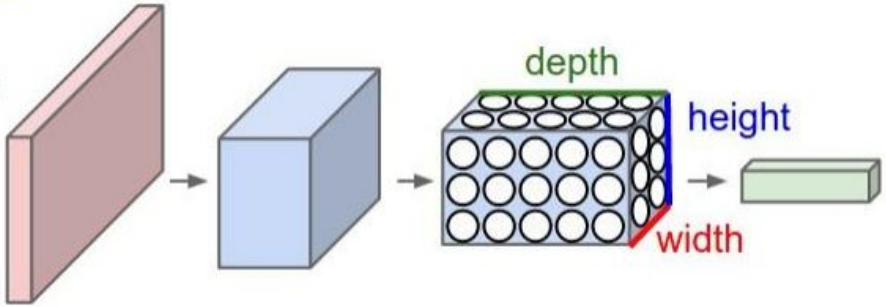


Convnets



Layers used to build ConvNets:

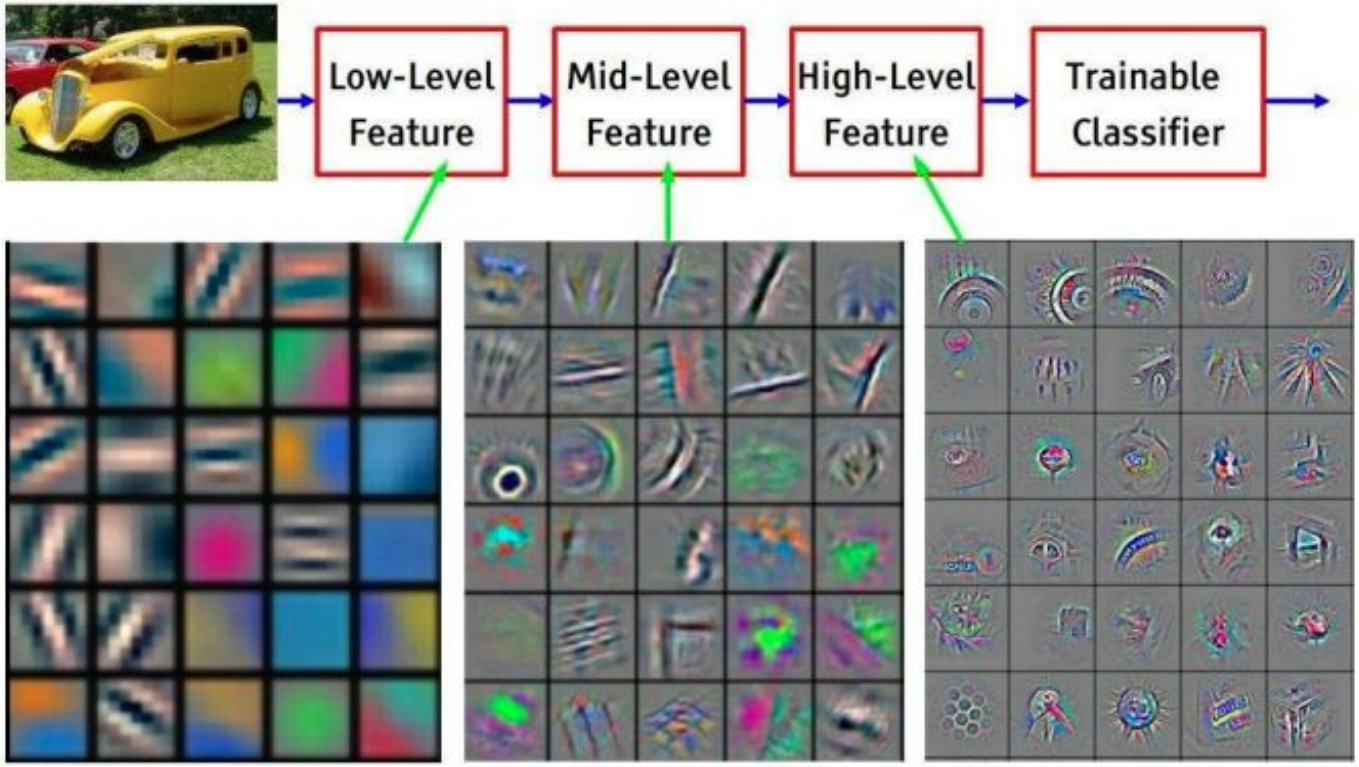
- a stacked sequence of layers. 3 main types
- Convolutional Layer, Pooling Layer, and Fully-Connected Layer



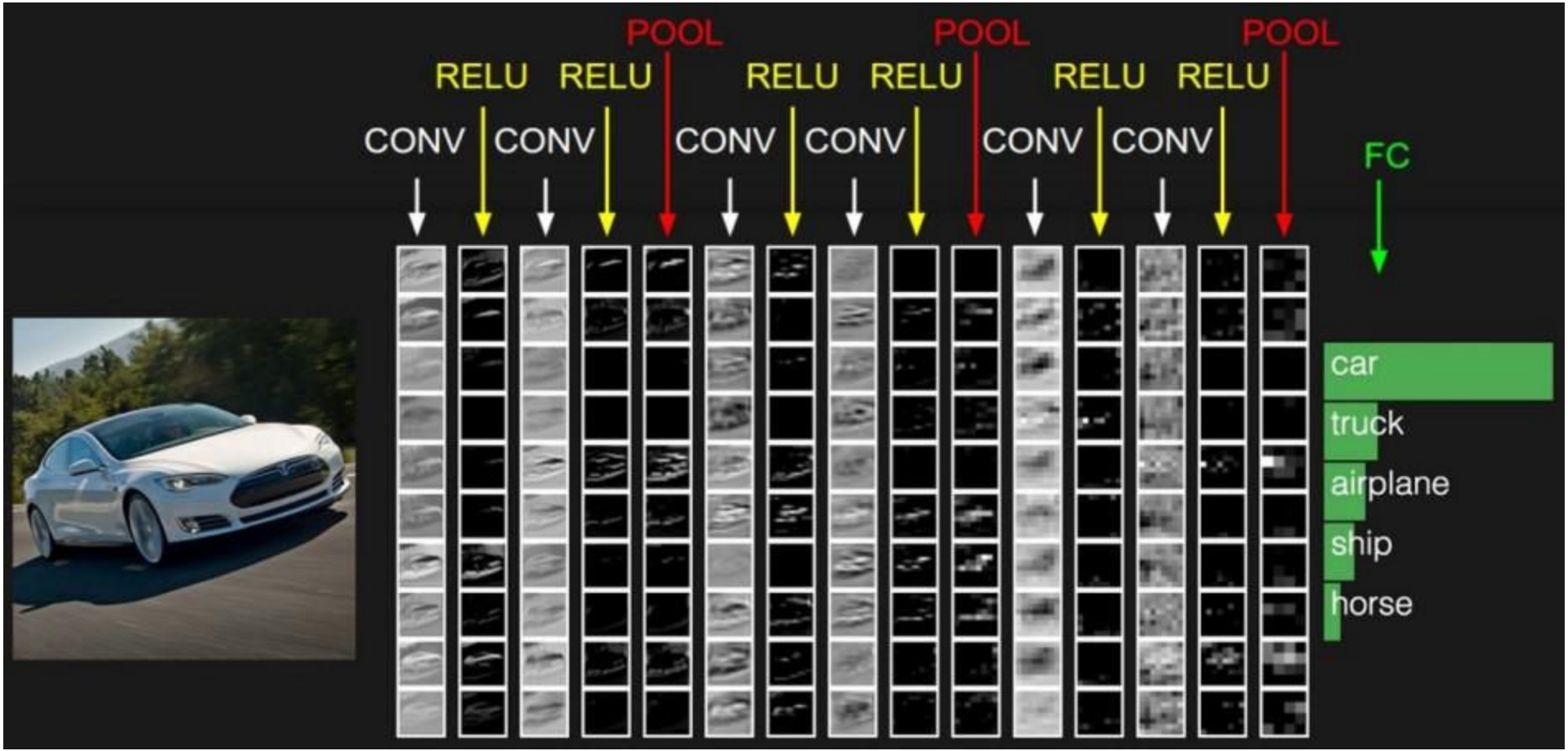
- every layer of a ConvNet transforms one volume of activations to another through a differentiable function.

INPUT -> [[CONV -> RELU]*N -> POOL?]*M ->[FC -> RELU]*K -> FC

- N >= 0 (and usually N <= 3), M >= 0, K >= 0

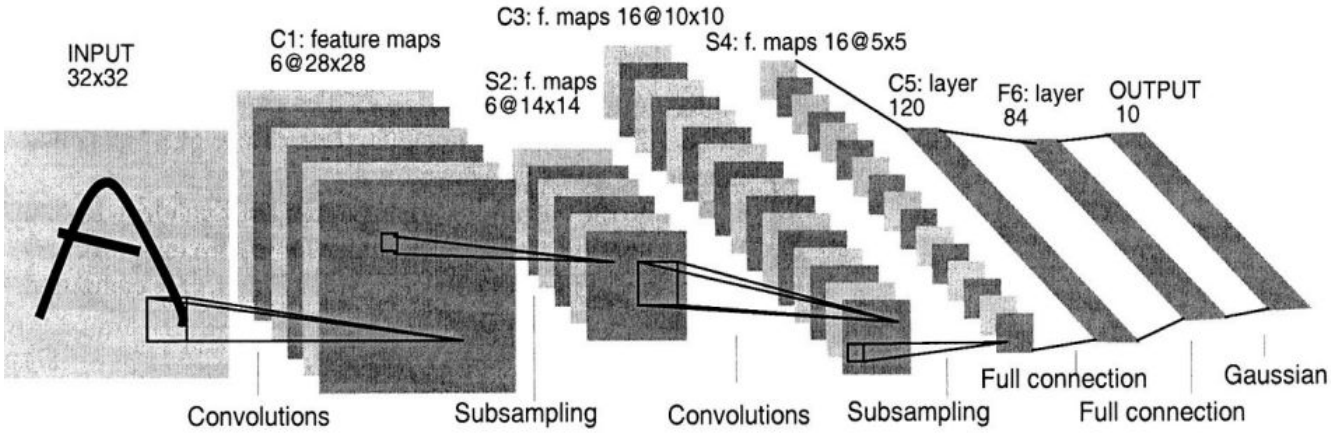


Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]


POOL
POOL
POOL
RELU
RELU
RELU
RELU
RELU
RELU
CONV
CONV
CONV
CONV
CONV
CONV
FC
car
truck
airplane
ship
horse

The architecture of LeNet5

1998, Yann LeCun



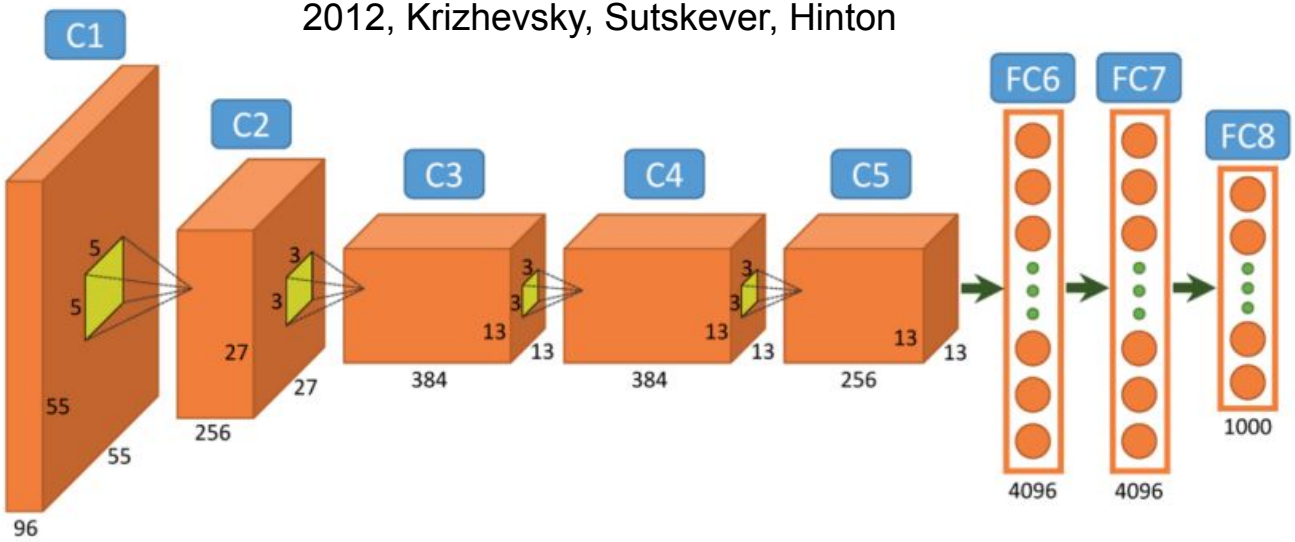
First successful convolutional neural network for handwritten digit recognition (MNIST).
Reduce parameters compared to fully connected nets, exploit spatial locality.

Architecture:

- Stacked **convolutions + pooling** followed by fully connected layers.
- Introduced weight sharing, local receptive fields.

A Common Architecture: AlexNet

2012, Krizhevsky, Sutskever, Hinton

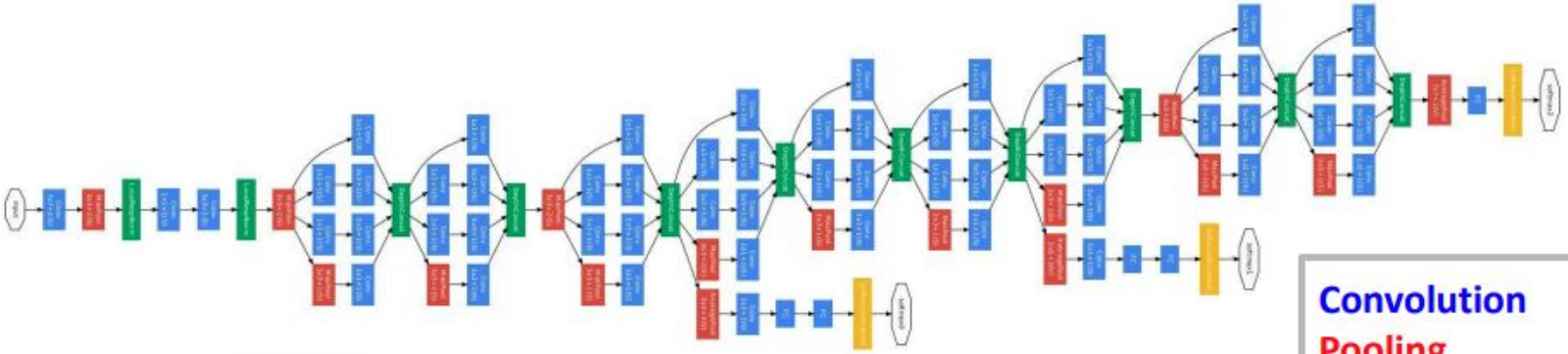


Winning **ImageNet 2012** by a huge margin (15.3% top-5 error vs ~26% previous).
Showed CNNs + GPUs could scale to large datasets and crush hand-crafted features (SIFT, HOG).
Architecture:

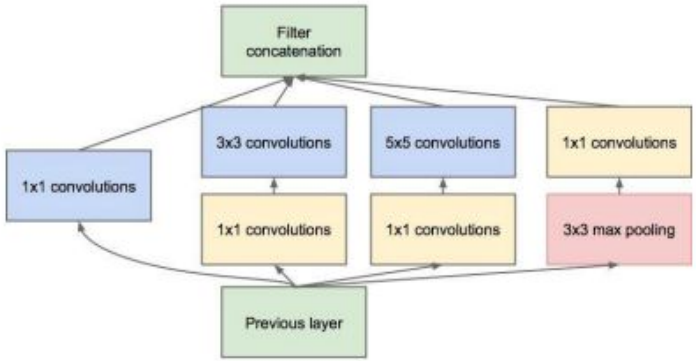
- 8 layers (5 conv, 3 fully connected).
- Introduced **ReLU** (faster training than sigmoid/tanh).
- Used **Dropout** (to reduce overfitting).
- Trained on **GPUs** (massively sped up deep learning).

GoogLeNet

[Szegedy et al., 2014]



Convolution
Pooling
Softmax
Other



Inception module

Inception modules: parallel convs (1×1, 3×3, 5×5) + pooling → concatenated.
Used **1×1 convolutions** as **bottleneck layers** to reduce computation.

ILSVRC 2014 winner (6.7% top 5 error)

Large Scale Visual Recognition Challenge

ResNet

2015, Kaiming He et al

Showed **depth = power**, but training very deep nets is hard.
Residual learning solved optimization difficulties.

- Introduced **Residual connections (skip connections)**: alleviates **vanishing gradient problem**, enabling ultra-deep training.
- Up to **152 layers** deep., improved accuracy.
- Winner ILSVRC 2015 with 3.5% (top-5) beat human-level performance (5%) in object recognition.

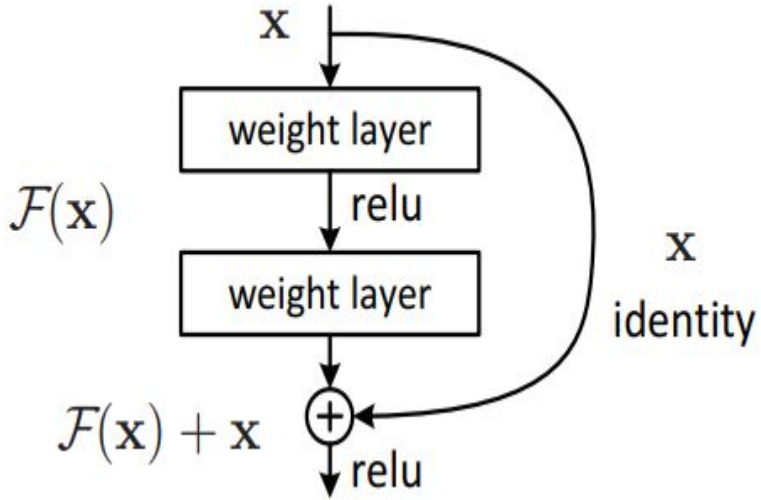
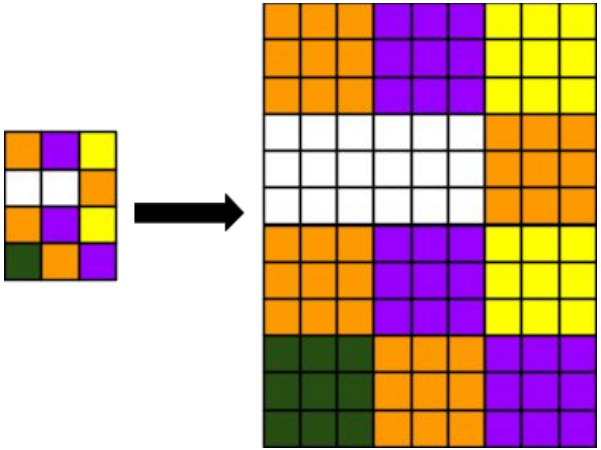


Image Segmentation

Upsampling: Resize feature map to larger size

- **Nearest neighbor**: copy nearest pixel value.
- **Bilinear**: weighted average of 4 nearest pixels.



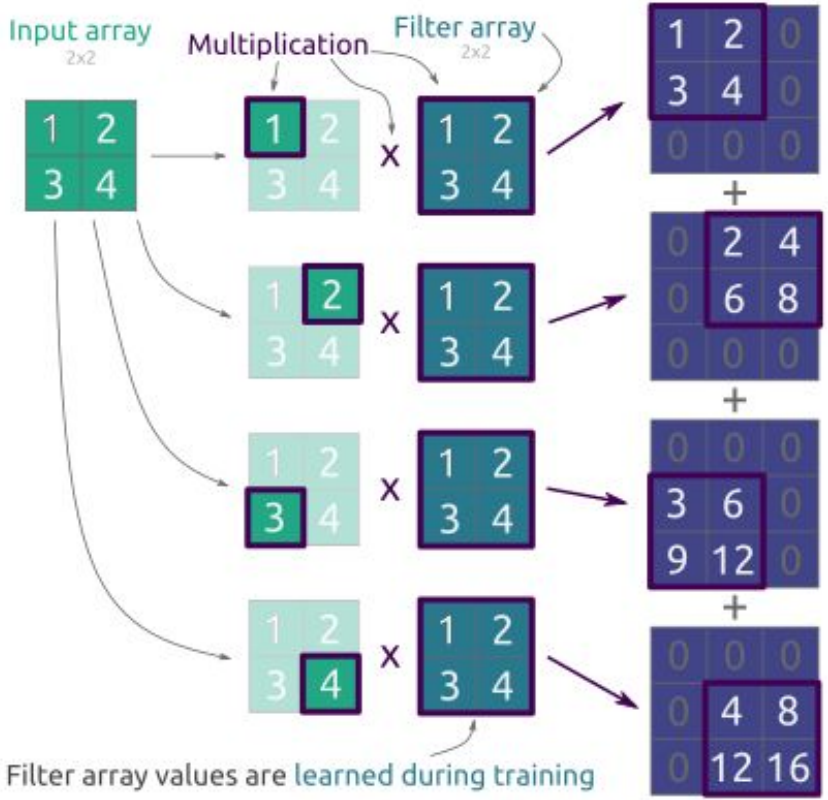
Transposed Convolution (Deconvolution):

Convolution = (N-F+2P)/S+1

Deconv/TC = (N−1)S−2P+F

Transposed Convolutions

"Smart" upsampling



A transposed convolution is used to **create an output array larger than the input array**

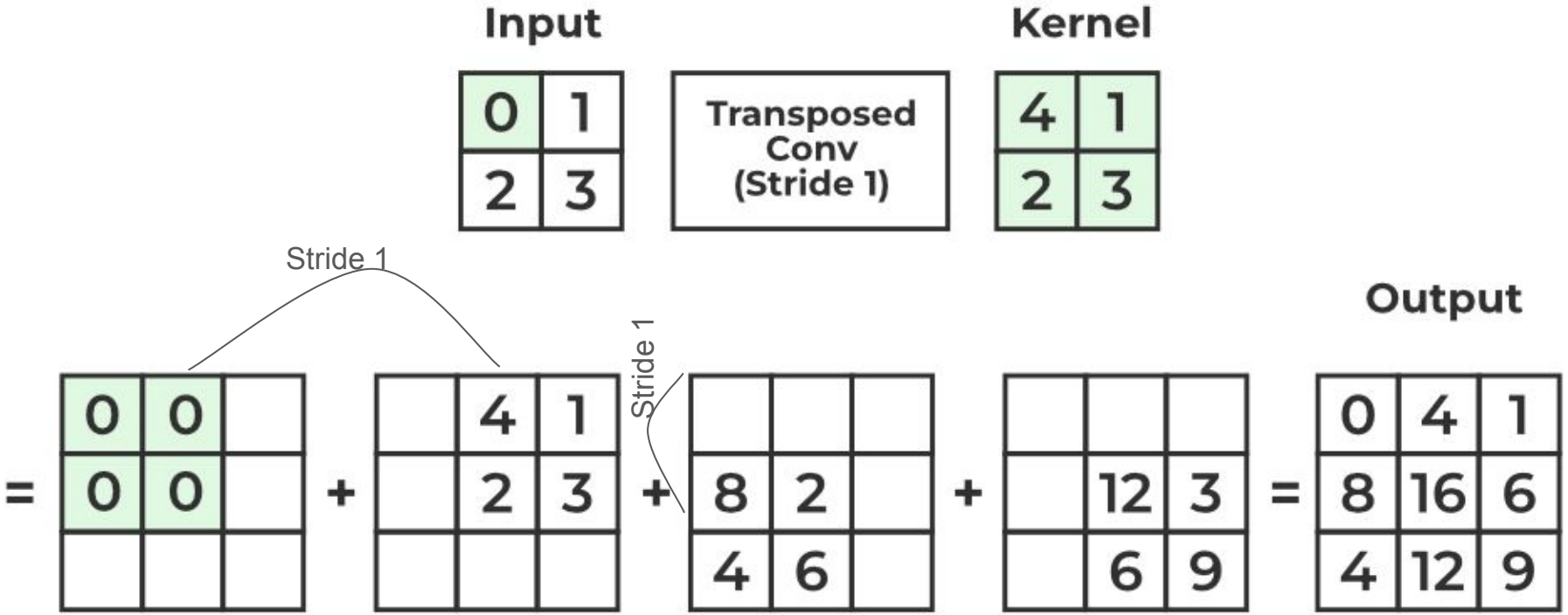
Output array
3x3

Sums =

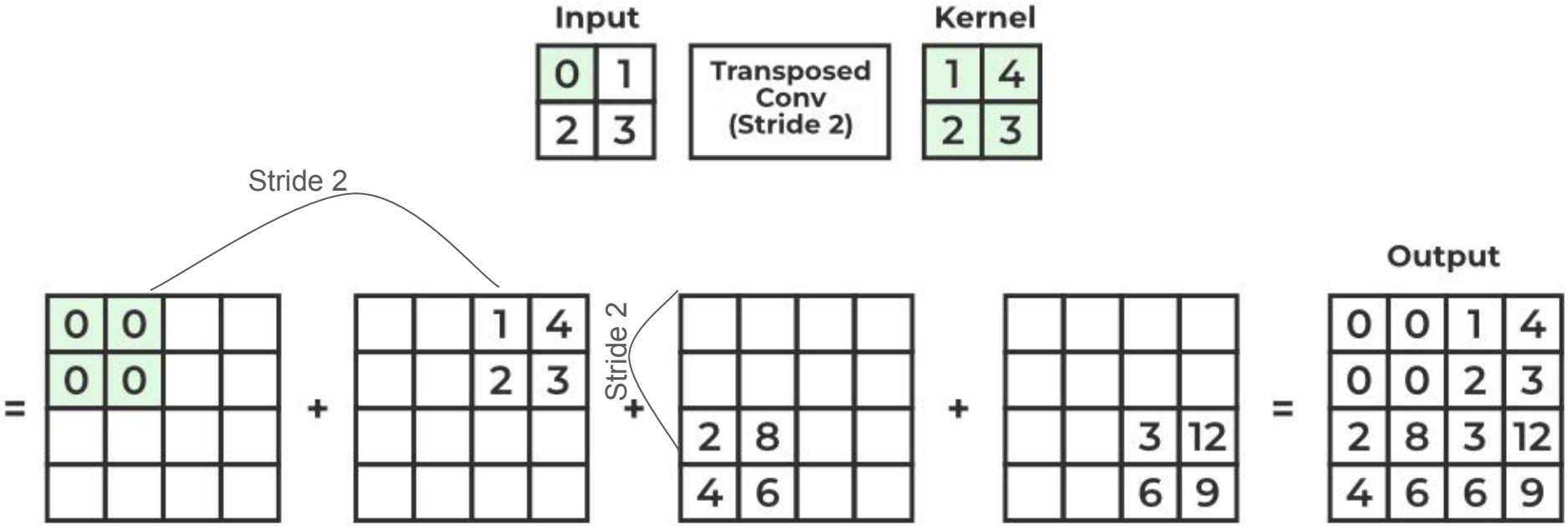
1	4	4
6	20	16
9	24	16

Transposed convolutions have hyperparameters, such as filter size, padding, and stride, depending on the goal

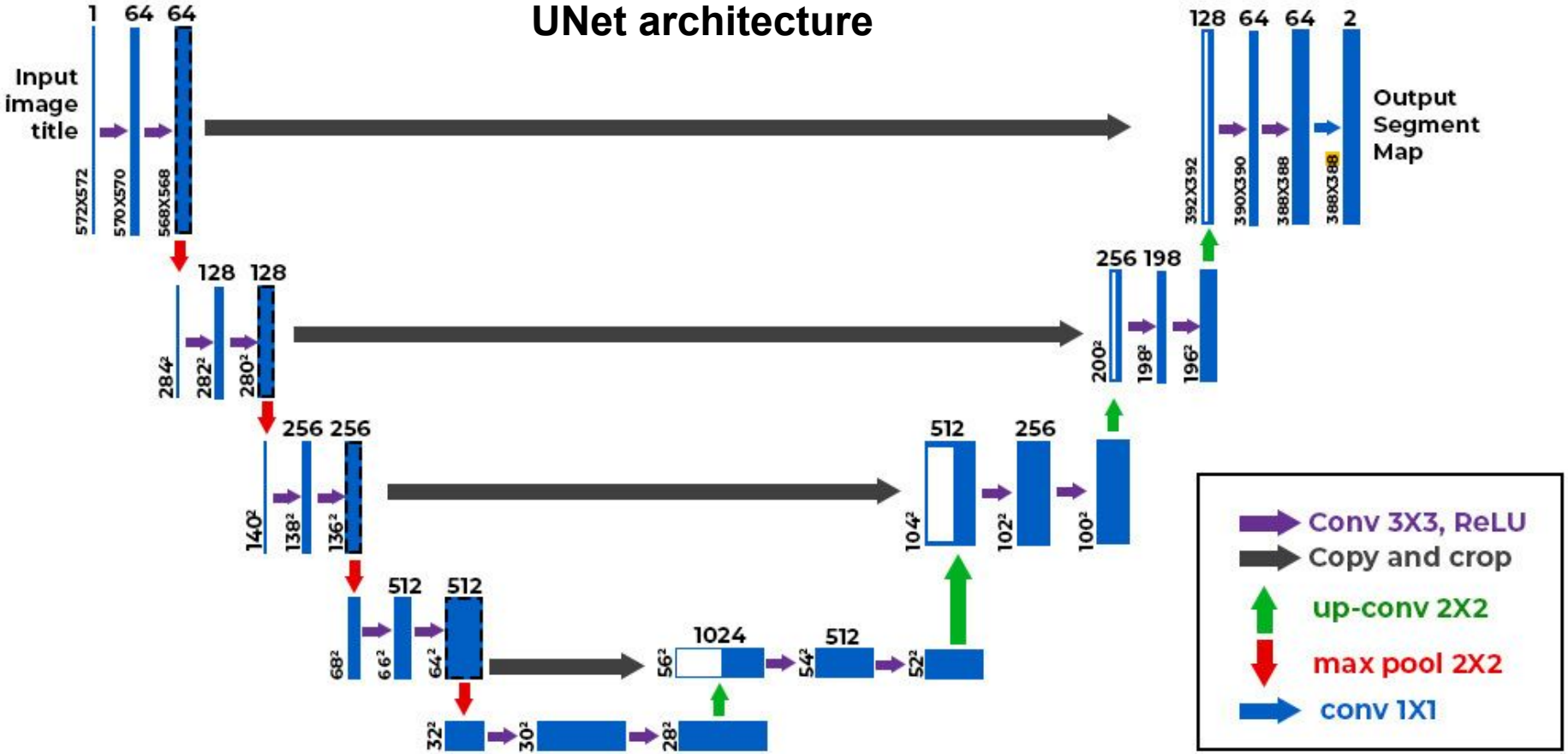
Out_size = (N−1)S−2P+F = (2-1)*1-0+2 = 3



Out_size = (N−1)S−2P+F = (2-1)*2-0+2 = 4



UNet architecture

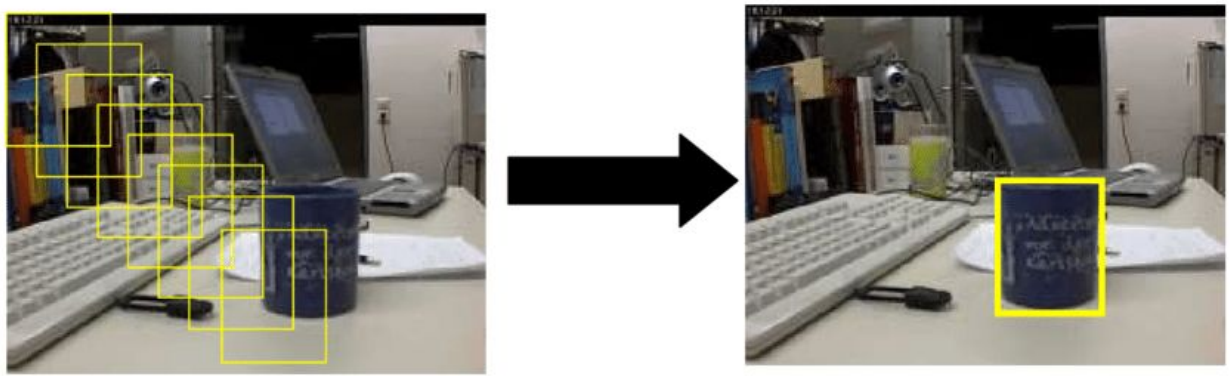


Object recognition in images

Traditional Object recognition process
Suppose the image is 1000 × 600 pixels.
Use a 64 × 64 detection window, stride 8 pixels.

- Horizontal positions ≈ (1000−64)/8 ≈ 117
- Vertical positions ≈ (600−64)/8 ≈ 67
- Windows at one scale ≈ 117 × 67 ≈ **7,800**.

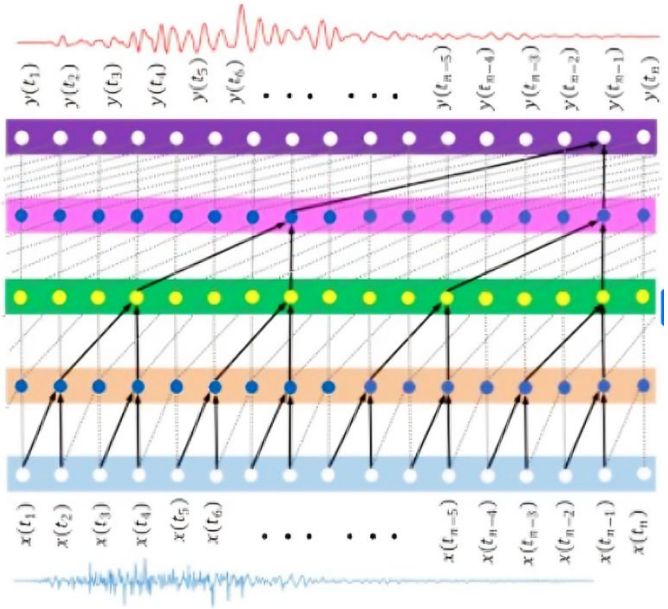
Now test 10 different window sizes/scales → **~78,000** windows.



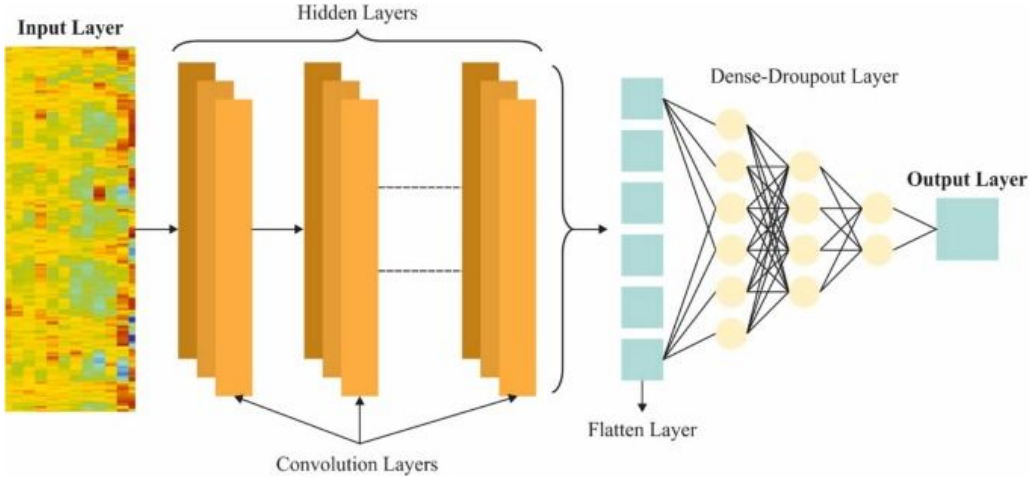
R-CNN (Regions with CNN features)

- Oversegment the image into superpixels.
- Merge neighboring superpixels that look similar (color, texture, size, fill).
- Every merge → a candidate box.
- After deduplication you keep ≈ 2000 high-quality proposals.

1D CNN



1D CNNs on raw wav samples



1D CNNs on processed speech vectors

Back Propagation in CNNs

Chain rule review:

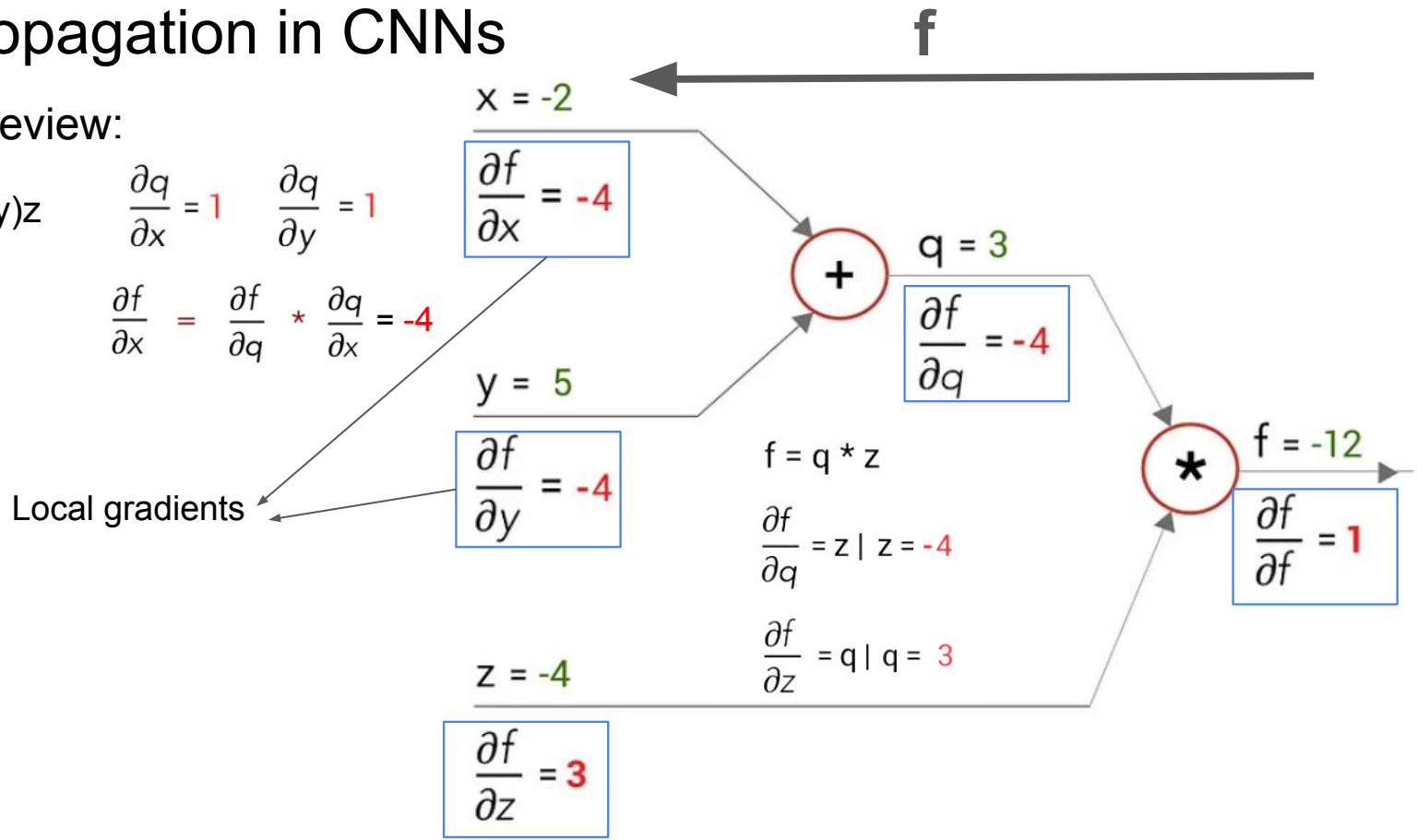
f(x,y,z) = (x + y)z

Let q=x+y

f=q*z

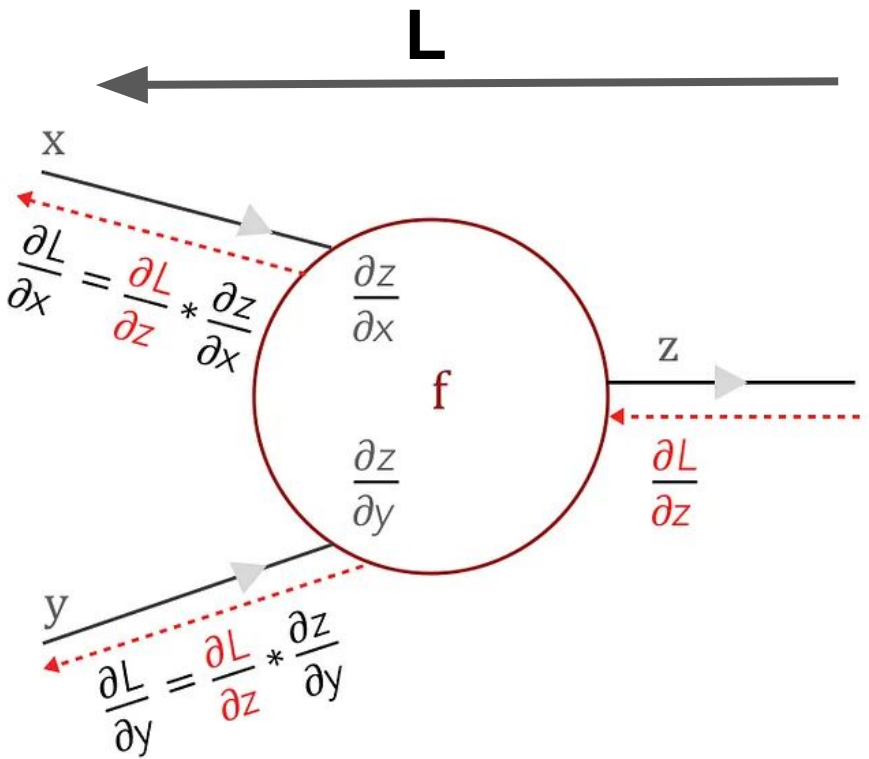
$\frac{\partial q}{\partial x} = 1$ $\frac{\partial q}{\partial y} = 1$

$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} * \frac{\partial q}{\partial x} = -4$

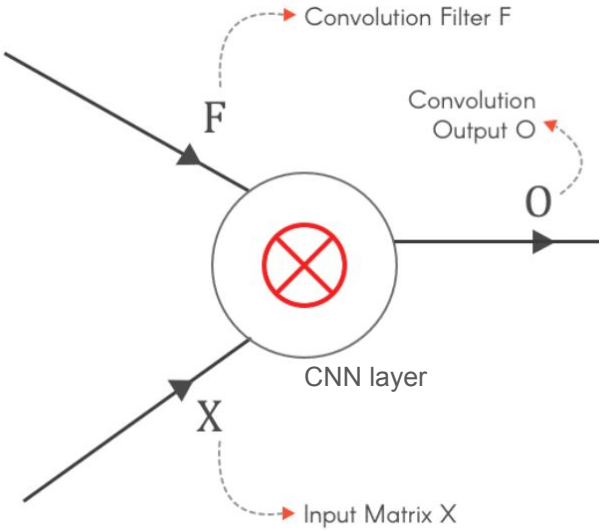


$\frac{\partial z}{\partial x}$ & $\frac{\partial z}{\partial y}$ *are local gradients*

$\frac{\partial L}{\partial z}$ *is the loss from the previous layer which has to be backpropagated to other layers*



Convolution example



$$\begin{bmatrix} O_{11} & O_{12} \\ O_{21} & O_{22} \end{bmatrix}$$

Output O

$$= \text{Convolution}\left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix} \right)$$

Input X

Filter F

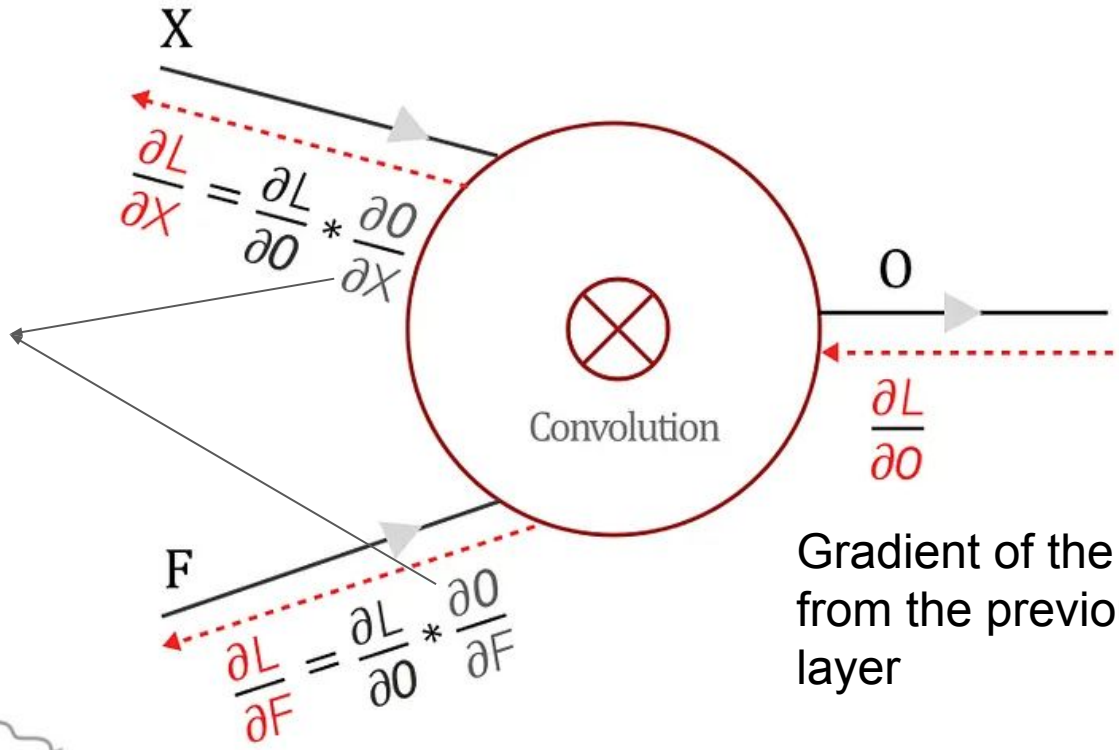
$$O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$$

$$O_{12} = X_{12}F_{11} + X_{13}F_{12} + X_{22}F_{21} + X_{23}F_{22}$$

$$O_{21} = X_{21}F_{11} + X_{22}F_{12} + X_{31}F_{21} + X_{32}F_{22}$$

$$O_{22} = X_{22}F_{11} + X_{23}F_{12} + X_{32}F_{21} + X_{33}F_{22}$$

Gradients



Gradient of the loss from the previous layer

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial O} * \frac{\partial O}{\partial F}$$

Gradient to update Filter F

Loss Gradient from previous layer

Local Gradients

Gradients to update filter/kernel

given $O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$

Local gradients $\frac{\partial O_{11}}{\partial F_{11}} = X_{11}$ $\frac{\partial O_{11}}{\partial F_{12}} = X_{12}$ $\frac{\partial O_{11}}{\partial F_{21}} = X_{21}$ $\frac{\partial O_{11}}{\partial F_{22}} = X_{22}$

For every element of F

$$\frac{\partial L}{\partial F_i} = \sum_{k=1}^{M} \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial F_i}$$

Loss gradient for kth output
* local gradient of kth output

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{11}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{11}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{11}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{11}}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{12}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{12}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{12}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{12}}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{21}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{21}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{21}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{21}}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{22}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{22}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{22}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{22}}$$

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * X_{11} + \frac{\partial L}{\partial O_{12}} * X_{12} + \frac{\partial L}{\partial O_{21}} * X_{21} + \frac{\partial L}{\partial O_{22}} * X_{22}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * X_{12} + \frac{\partial L}{\partial O_{12}} * X_{13} + \frac{\partial L}{\partial O_{21}} * X_{22} + \frac{\partial L}{\partial O_{22}} * X_{23}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * X_{21} + \frac{\partial L}{\partial O_{12}} * X_{22} + \frac{\partial L}{\partial O_{21}} * X_{31} + \frac{\partial L}{\partial O_{22}} * X_{32}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * X_{22} + \frac{\partial L}{\partial O_{12}} * X_{23} + \frac{\partial L}{\partial O_{21}} * X_{32} + \frac{\partial L}{\partial O_{22}} * X_{33}$$

Gradients to update filter

This is represented as a *convolution* operation between

input X and **loss gradient ∂L/∂O**

$$\begin{bmatrix} \frac{\partial L}{\partial F_{11}} & \frac{\partial L}{\partial F_{12}} \\ \frac{\partial L}{\partial F_{21}} & \frac{\partial L}{\partial F_{22}} \end{bmatrix} = \text{Convolution} \left(\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}, \begin{bmatrix} \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \end{bmatrix} \right)$$

where

$$\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} = \text{Input X} \qquad \begin{bmatrix} \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \end{bmatrix} = \frac{\partial L}{\partial O}$$ Loss gradient from previous layer

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * X_{11} + \frac{\partial L}{\partial O_{12}} * X_{12} + \frac{\partial L}{\partial O_{21}} * X_{21} + \frac{\partial L}{\partial O_{22}} * X_{22}$$

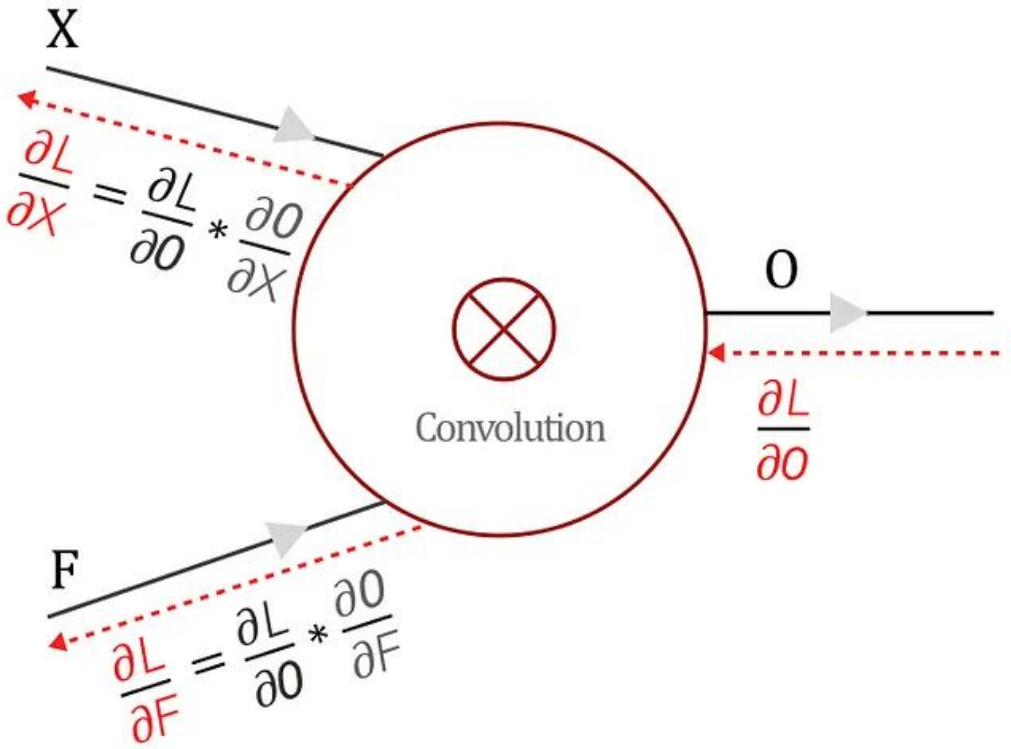
$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * X_{12} + \frac{\partial L}{\partial O_{12}} * X_{13} + \frac{\partial L}{\partial O_{21}} * X_{22} + \frac{\partial L}{\partial O_{22}} * X_{23}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * X_{21} + \frac{\partial L}{\partial O_{12}} * X_{22} + \frac{\partial L}{\partial O_{21}} * X_{31} + \frac{\partial L}{\partial O_{22}} * X_{32}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * X_{22} + \frac{\partial L}{\partial O_{12}} * X_{23} + \frac{\partial L}{\partial O_{21}} * X_{32} + \frac{\partial L}{\partial O_{22}} * X_{33}$$

Gradients

$\partial L/\partial X = ?$



Finding ∂L/∂X

given $O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$

Local gradients $\frac{\partial O_{11}}{\partial X_{11}} = F_{11} \quad \frac{\partial O_{11}}{\partial X_{12}} = F_{12} \quad \frac{\partial O_{11}}{\partial X_{21}} = F_{21} \quad \frac{\partial O_{11}}{\partial X_{22}} = F_{22}$

For every element of X_i

$$\frac{\partial L}{\partial X_i} = \sum_{k=1}^{M} \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial X_i}$$

$$\Longrightarrow$$

$$\frac{\partial L}{\partial X_{11}} = \frac{\partial L}{\partial O_{11}} * F_{11}$$

$$\frac{\partial L}{\partial X_{12}} = \frac{\partial L}{\partial O_{11}} * F_{12} + \frac{\partial L}{\partial O_{12}} * F_{11}$$

$$\frac{\partial L}{\partial X_{13}} = \frac{\partial L}{\partial O_{12}} * F_{12}$$

$$\frac{\partial L}{\partial X_{21}} = \frac{\partial L}{\partial O_{11}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{11}$$

$$\frac{\partial L}{\partial X_{22}} = \frac{\partial L}{\partial O_{11}} * F_{22} + \frac{\partial L}{\partial O_{12}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{12} + \frac{\partial L}{\partial O_{22}} * F_{11}$$

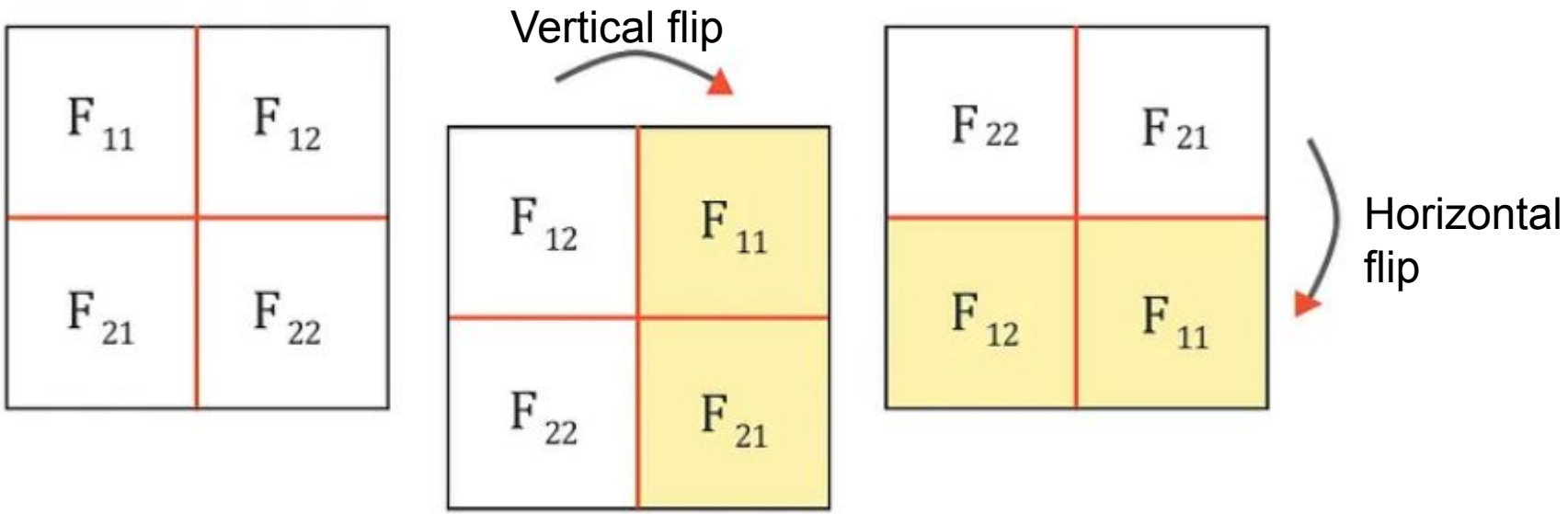
$$\frac{\partial L}{\partial X_{23}} = \frac{\partial L}{\partial O_{12}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{12}$$

$$\frac{\partial L}{\partial X_{31}} = \frac{\partial L}{\partial O_{21}} * F_{21}$$

$$\frac{\partial L}{\partial X_{32}} = \frac{\partial L}{\partial O_{21}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{21}$$

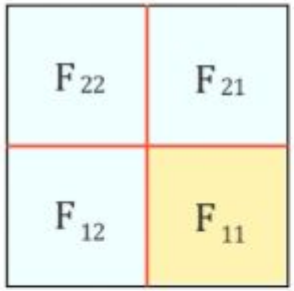
$$\frac{\partial L}{\partial X_{33}} = \frac{\partial L}{\partial O_{22}} * F_{22}$$

Gradients of input as convolution



Input gradient computation with flipped filter and full convolution

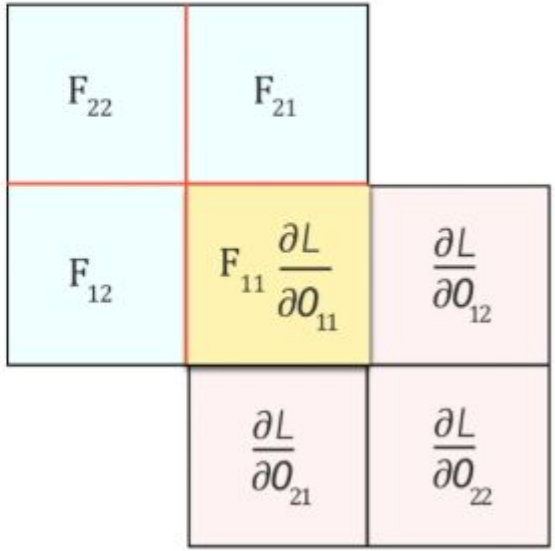
Bottom to top &
Right to left



Filter F

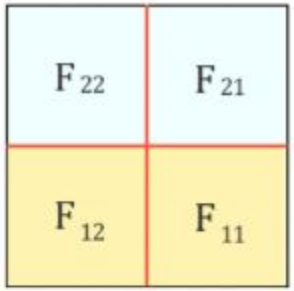
Loss Gradient $\frac{\partial L}{\partial O}$

$$\frac{\partial L}{\partial X_{11}} = F_{11} * \frac{\partial L}{\partial O_{11}}$$

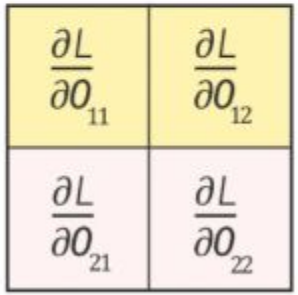


Input gradient computation with flipped filter and full convolution

Bottom to top &
Right to left

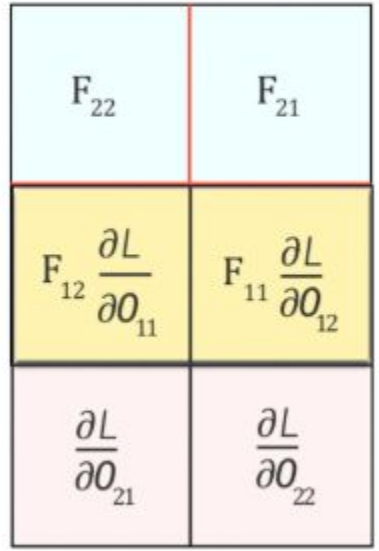


Filter F



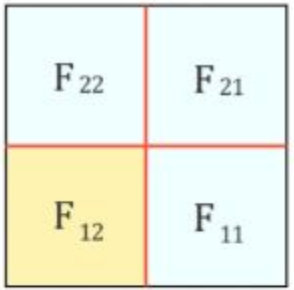
Loss Gradient $\frac{\partial L}{\partial O}$

$$\frac{\partial L}{\partial X_{12}} = F_{12} * \frac{\partial L}{\partial O_{11}} + F_{11} * \frac{\partial L}{\partial O_{12}}$$



Input gradient computation with flipped filter and full convolution

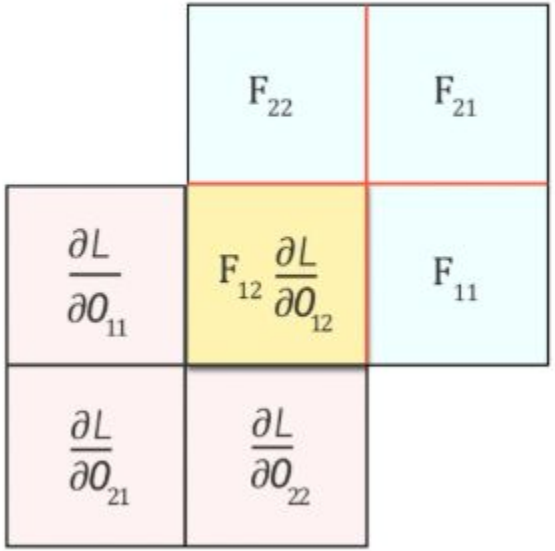
Bottom to top &
Right to left



Filter F

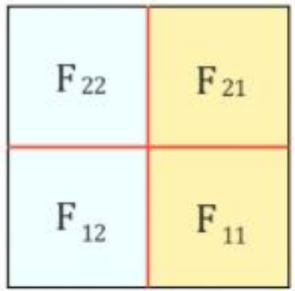
Loss Gradient $\frac{\partial L}{\partial O}$

$$\frac{\partial L}{\partial X_{13}} = F_{12} * \frac{\partial L}{\partial O_{12}}$$

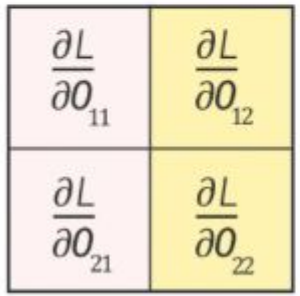


Input gradient computation with flipped filter and full convolution

Bottom to top &
Right to left

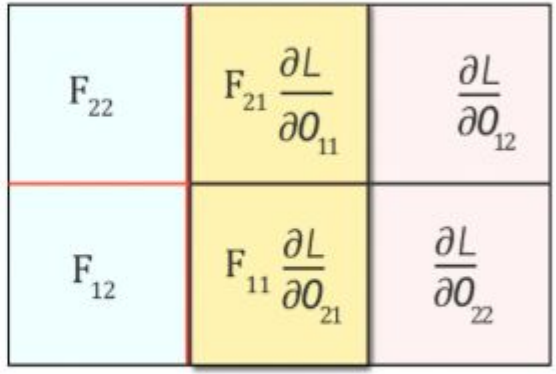


Filter F



Loss Gradient $\frac{\partial L}{\partial O}$

$$\frac{\partial L}{\partial X_{21}} = F_{21} * \frac{\partial L}{\partial O_{11}} + F_{11} * \frac{\partial L}{\partial O_{21}}$$



Input gradient computation with flipped filter and full convolution

Bottom to top &
Right to left

F_{22}	F_{21}
F_{12}	F_{11}

Filter F

$\frac{\partial L}{\partial O_{11}}$	$\frac{\partial L}{\partial O_{12}}$
$\frac{\partial L}{\partial O_{21}}$	$\frac{\partial L}{\partial O_{22}}$

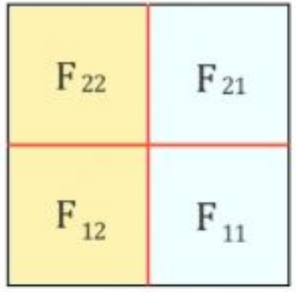
Loss Gradient $\frac{\partial L}{\partial O}$

$$\frac{\partial L}{\partial X_{22}} = F_{22} * \frac{\partial L}{\partial O_{11}} + F_{21} * \frac{\partial L}{\partial O_{12}} + F_{12} * \frac{\partial L}{\partial O_{21}} + F_{11} * \frac{\partial L}{\partial O_{22}}$$

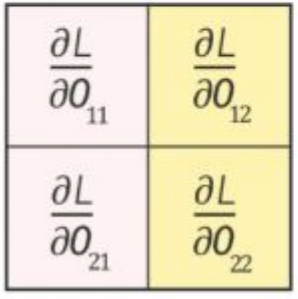
$F_{22}\frac{\partial L}{\partial O_{11}}$	$F_{21}\frac{\partial L}{\partial O_{12}}$
$F_{12}\frac{\partial L}{\partial O_{21}}$	$F_{11}\frac{\partial L}{\partial O_{22}}$

Input gradient computation with flipped filter and full convolution

Bottom to top &
Right to left

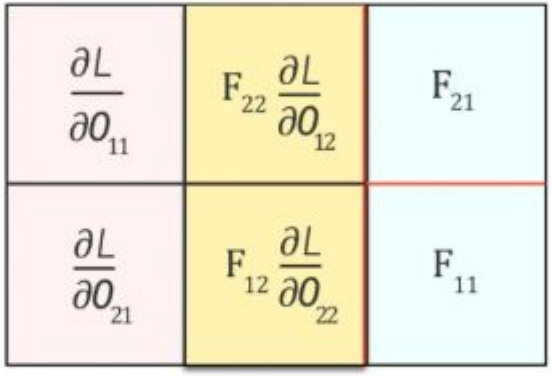


Filter F



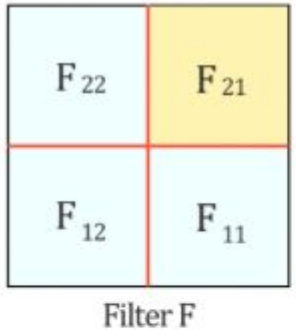
Loss Gradient $\frac{\partial L}{\partial O}$

$$\frac{\partial L}{\partial X_{23}} = F_{22} * \frac{\partial L}{\partial O_{12}} + F_{12} * \frac{\partial L}{\partial O_{22}}$$



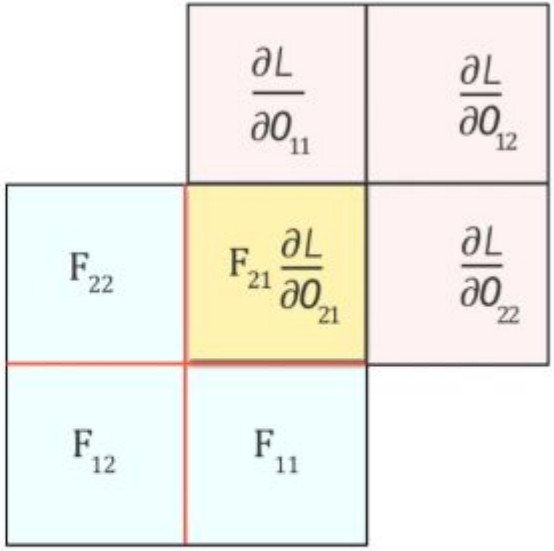
Input gradient computation with flipped filter and full convolution

Bottom to top &
Right to left



Filter F

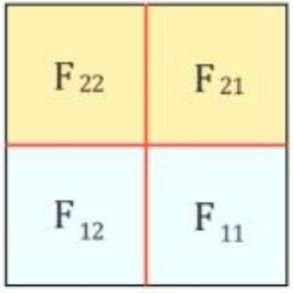
Loss Gradient $\frac{\partial L}{\partial O}$



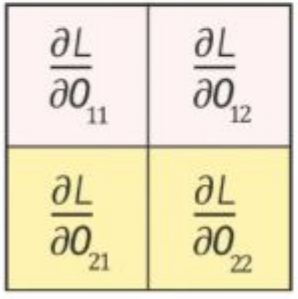
$$\frac{\partial L}{\partial X_{31}} = F_{21} * \frac{\partial L}{\partial O_{21}}$$

Input gradient computation with flipped filter and full convolution

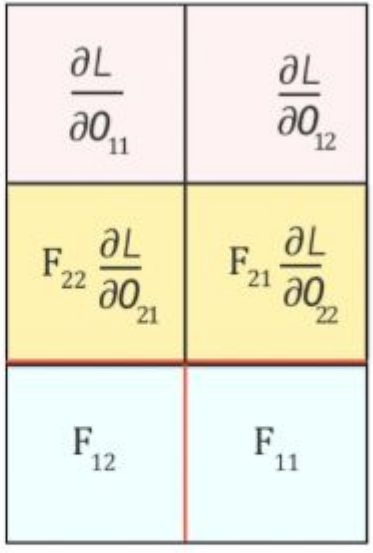
Bottom to top &
Right to left



Filter F



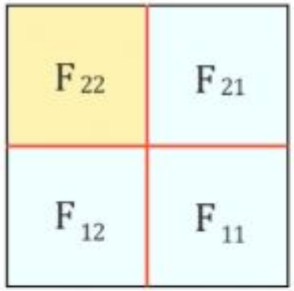
Loss Gradient $\frac{\partial L}{\partial O}$



$$\frac{\partial L}{\partial X_{32}} = F_{22} * \frac{\partial L}{\partial O_{21}} + F_{21} * \frac{\partial L}{\partial O_{22}}$$

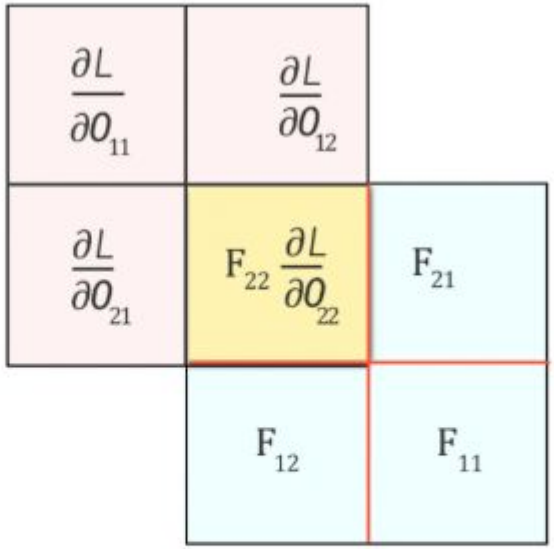
Input gradient computation with flipped filter and full convolution

Bottom to top &
Right to left



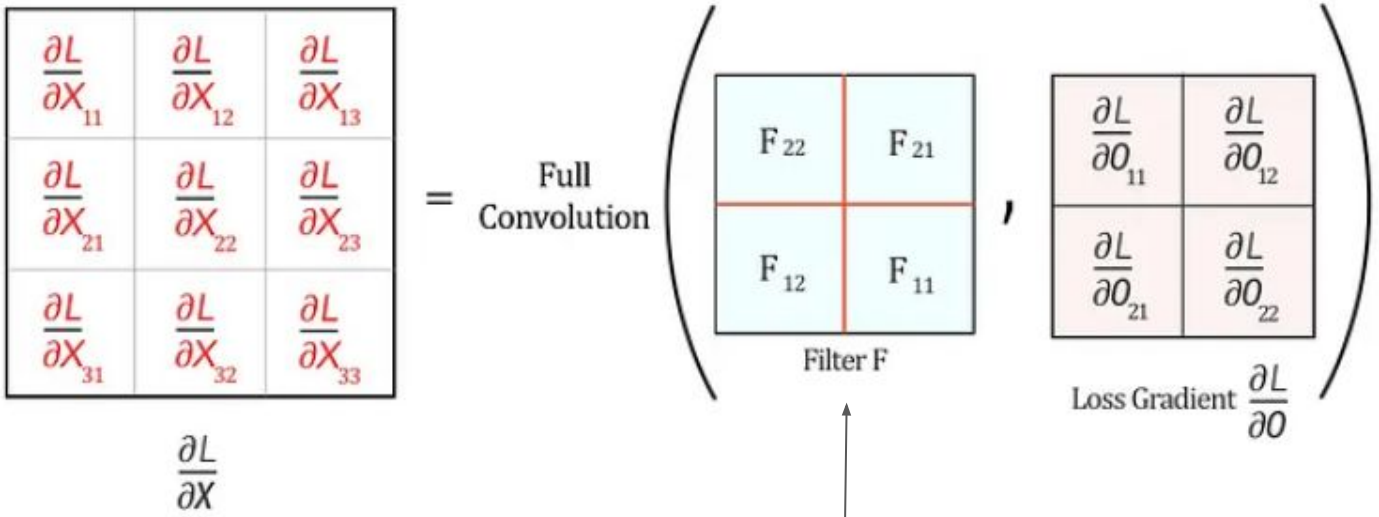
Filter F

Loss Gradient $\frac{\partial L}{\partial O}$



$$\frac{\partial L}{\partial X_{33}} = F_{22} * \frac{\partial L}{\partial O_{22}}$$

Input gradient computation with flipped filter and full convolution



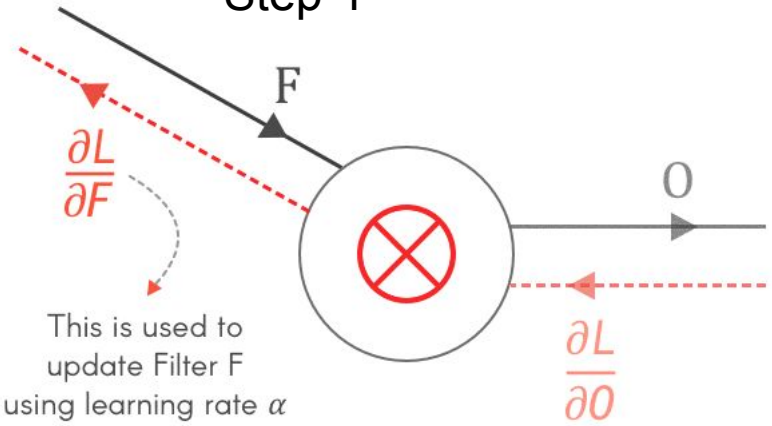
$$\frac{\partial L}{\partial X} = \text{Full Convolution}\left(180^{\circ}\text{rotated Filter } F, \text{Loss Gradient } \frac{\partial L}{\partial O}\right)$$

Full convolution: slide filter from Bottom to top & Right to left

Summary

- Forward pass the input,
- Find the output layer gradient and
- Repeat following steps for each layer

Step-1

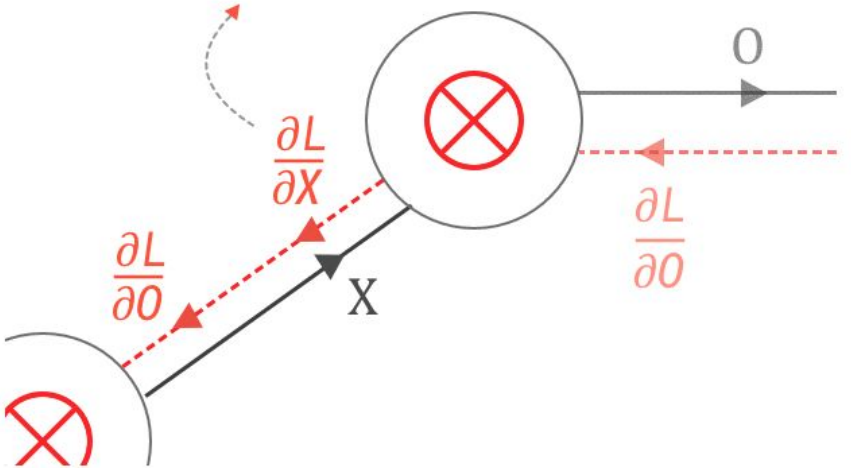


$$F_{updated} = F - \alpha \frac{\partial L}{\partial F}$$

$$\frac{\partial L}{\partial F} = \text{Convolution}\left(\text{Input } X, \text{ Loss gradient } \frac{\partial L}{\partial O}\right)$$

Step-2

Since X is the output of the previous layer, $\partial L/\partial X$ becomes the loss gradient for the previous layer



$$\frac{\partial L}{\partial X} = \text{Full Convolution}\left(180^{\circ}\text{ rotated Filter F}, \text{Loss Gradient } \frac{\partial L}{\partial O}\right)$$

Both the Forward pass and the Backpropagation of a Convolutional layer are Convolutions